time-series: the Time Series program — user guide

time-series-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Time Series covers moving averages, exponential smoothing, autocorrelation, ARIMA basics, stationarity tests, change points, forecasting accuracy and Kalman filtering. This guide is about `time-series`, the model as a self-contained program for Windows and Linux: 77 capabilities in 14 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/time-series-ai999x/); the Python package (`pip install time-series-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Time Series	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-time-series-1.4.0-windows-x86_64.zip
Desktop app: Time Series	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-time-series-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Time Series in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-time-series-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-time-series.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-time-series-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-time-series-1.4.0-linux-x86_64.tar.gz
./ai999x-time-series-1.4.0-linux-x86_64/ai999x-time-series
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `time-series` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source ~/.bashrc
time-series --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User") + ";$D", "User")
time-series --version
```

In your browser

Open https://www.ai999x.com/models/time-series-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install time-series-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ time-series --version
time-series-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ time-series ask "Find the 3-period simple moving average of [10, 12, 14, 13, 15]."
Read as: Simple moving average
sma = [12.0,13.0,14.0]
last = 14
count = 3
  by: Simple moving average  (mean of each window of k consecutive values)
  assumes: 1 <= k <= n
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ time-series search "simple moving average"
simple_moving_average                Simple moving average  (moving averages)
exponential_moving_average           Exponential moving average  (moving averages)
centred_moving_average               Centred moving average  (moving averages)
linearly_weighted_moving_average     Linearly weighted moving average  (moving averages)
simple_exponential_smoothing         Simple exponential smoothing  (exponential smoothing)
compound_average_growth_rate         Compound average growth rate of a series  (transformations and growth)
sample_autocorrelation               Sample autocorrelation at lag k  (autocorrelation)
sample_autocovariance                Sample autocovariance at lag k  (autocorrelation)
… 8 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ time-series list
ar1_forecast                         AR(1) forecast towards the mean
ar1_properties                       Mean, variance and autocorrelation of an AR(1) process
ar2_stationarity                     Stationarity of an AR(2) process
ar2_theoretical_acf                  Theoretical autocorrelations of an AR(2) process
ar_forecast                          Forecasts of an AR(p) model
ar_least_squares                     AR(p) estimation by conditional least squares
ar_yule_walker                       AR(p) estimation by Yule-Walker
arima_p10_forecast                   Forecasts of an ARIMA(p,1,0) model
… 69 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ time-series info simple_moving_average
Simple moving average
  mean of each window of k consecutive values
  in   x        the series, oldest first (vector) [1]
  in   k        the window length (number of periods) (integer) [1]
  out  sma      the moving averages, one per complete window (the first covers x_1..x_k) [1]
  out  last     the latest moving average (forecast of the next period) [1]
  out  count    number of moving averages (n - k + 1) [1]
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ time-series solve simple_moving_average "x=[10, 12, 14, 13, 15]" k=3
sma = [12.0,13.0,14.0]
last = 14
count = 3
  by: Simple moving average  (mean of each window of k consecutive values)
  assumes: 1 <= k <= n
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ time-series solve simple_moving_average "x=[10, 12, 14, 13, 15]" k=3 --json
{"outcome":"SOLVED","message":"","value":14.0,"unit":"","solved_for":"last","values":[["sma",[12.0,13.0,14.0],""],
["last",14.0,""],["count",3,""]],"target":"last","phrase":"the moving averages, one per complete window (the first covers
x_1..x_k)","assumptions":["1 <= k <= n"],"name":"Simple moving average","equation":"mean of each window of k consecutive values"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ time-series solve simple_moving_average "x=[10, 12, 14, 13, 15]"
NEEDS_INPUT: simple_moving_average needs k (the window length (number of periods)). Example: solve('simple_moving_average', x='10,
12, 14, 13, 15', k=3).
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ time-series figures simple_moving_average "x=[10, 12, 14, 13, 15]" k=3 --out .
[
  {
    "slug": "simple-moving-average-smooth",
    "title": "Simple moving average trailing the data",
    "caption": "The series x = 10, 12, 14, 13, 15 with its simple moving average over k = 3 periods (each value placed at the end
of its window: 12, 13, 14) and, for comparison, the exponential moving average with the same span (dashed), returned by the model.
Your inputs: x = (list), k = 3.",
    "file": ".\\simple-moving-average-smooth.svg"
  },
  {
    "slug": "simple-moving-average-live-1",
    "title": "The latest moving average (forecast of the ne... as the window length (number of periods) varies",
    "caption": "The latest moving average (forecast of the ne... as k runs from 1 to 5, every other input as you entered it (5
solutions). Marked: your k = 3 gives last = 14. Your inputs: x = (list), k = 3.",
    "file": ".\\simple-moving-average-live-1.svg"
  },
  {
    "slug": "simple-moving-average-live-2",
    "title": "The moving averages, one per complete window..., as computed",
    "caption": "Every entry of sma as computed for your inputs (3 entries: 12, 13, 14). Your inputs: x = (list), k = 3.",
    "file": ".\\simple-moving-average-live-2.svg"
  },
  {
    "slug": "simple-moving-average-live-3",
    "title": "The latest moving average (forecast of the ne... as the series, oldest first, entry 5 varies",
    "caption": "The latest moving average (forecast of the ne... as x[5] runs from 3.75 to 26.25, every other input as you entered
it (29 solutions). Marked: your x[5] = 15 gives last = 14. Your inputs: x = (list), k = 3.",
    "file": ".\\simple-moving-average-live-3.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | time-series batch

`cases.jsonl`:

```
{"key": "simple_moving_average", "inputs": {"x": "[10, 12, 14, 13, 15]", "k": "3"}}
{"key": "holt_linear_trend", "inputs": {"x": "[10, 12, 15, 16, 19, 21]", "alpha": 0.5, "beta": 0.3}}
```

```
$ time-series batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":14.0,"unit":"","solved_for":"last","values":[["sma",[12.0,13.0,14.0],""],
["last",14.0,""],["count",3,""]],"target":"last","phrase":"the moving averages, one per complete window (the first covers
x_1..x_k)","assumptions":["1 <= k <= n"],"name":"Simple moving average","equation":"mean of each window of k consecutive values"}
{"outcome":"SOLVED","message":"","value":22.653285906249998,"unit":"","solved_for":"forecast","si":[["alpha",0.5],
["beta",0.3]],"values":[["forecast",22.653285906249998,""],["level",20.6553396875,""],["trend",1.9979462187499992,""],
["sse",7.492761855456635,""]],"target":"forecast","phrase":"the h-step-ahead forecast","assumptions":["0 < alpha, beta <= 1","l_0
= x_1, b_0 = x_2 - x_1 unless stated","trend form: beta multiplies (l_t - l_{t-1}) (the error-correction beta* of many
texts)"],"name":"Holt's linear trend method","equation":"l_t = alpha x_t + (1-alpha)(l_{t-1}+b_{t-1}); b_t = beta (l_t - l_{t-1})
+ (1-beta) b_{t-1}"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `time-series info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`time-series solve ... --json` prints one answer as one line of JSON. `time-series batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | time-series batch`. `time-series catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["time-series", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`time-series figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ time-series figures simple_moving_average "x=[10, 12, 14, 13, 15]" k=3 --out .
[
  {
    "slug": "simple-moving-average-smooth",
    "title": "Simple moving average trailing the data",
    "caption": "The series x = 10, 12, 14, 13, 15 with its simple moving average over k = 3 periods (each value placed at the end
of its window: 12, 13, 14) and, for comparison, the exponential moving average with the same span (dashed), returned by the model.
Your inputs: x = (list), k = 3.",
    "file": ".\\simple-moving-average-smooth.svg"
  },
  {
    "slug": "simple-moving-average-live-1",
    "title": "The latest moving average (forecast of the ne... as the window length (number of periods) varies",
    "caption": "The latest moving average (forecast of the ne... as k runs from 1 to 5, every other input as you entered it (5
solutions). Marked: your k = 3 gives last = 14. Your inputs: x = (list), k = 3.",
    "file": ".\\simple-moving-average-live-1.svg"
  },
  {
    "slug": "simple-moving-average-live-2",
    "title": "The moving averages, one per complete window..., as computed",
    "caption": "Every entry of sma as computed for your inputs (3 entries: 12, 13, 14). Your inputs: x = (list), k = 3.",
    "file": ".\\simple-moving-average-live-2.svg"
  },
  {
    "slug": "simple-moving-average-live-3",
    "title": "The latest moving average (forecast of the ne... as the series, oldest first, entry 5 varies",
    "caption": "The latest moving average (forecast of the ne... as x[5] runs from 3.75 to 26.25, every other input as you entered
it (29 solutions). Marked: your x[5] = 15 gives last = 14. Your inputs: x = (list), k = 3.",
    "file": ".\\simple-moving-average-live-3.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `time_series_ai999x.wasm` (https://www.ai999x.com/compute/rust/time_series_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("time_series_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `time-series info <id>` shows the same inputs and outputs in the terminal.

ARIMA models (7)

AR(1) forecast towards the mean — `ar1_forecast`

The h-step forecast mu + phi^h (x_n - mu) of a stationary AR(1) with mean mu, and (when the innovation variance is given) its forecast error variance and 95 % prediction interval.

$$\hat{x}_{n+h} = \mu + \phi^h (x_n - \mu),\ \ \mathrm{Var} = \sigma^2 \frac{1-\phi^{2h}}{1-\phi^2}$$

symbol		meaning	SI unit
`x`	in	the last observed value x_n	—
`phi`	in	AR coefficient, |phi| < 1	—
`mu`	in	the process mean	—
`h`	in	forecast horizon (default 1) (optional)	—
`sigma2`	in	innovation variance (optional, for the interval) (optional)	—
`forecast`	out	mu + phi^h (x_n - mu)	—
`variance`	out	sigma^2 (1 - phi^(2h)) / (1 - phi^2)	—

symbol		meaning	SI unit
`lower`	out	lower 95 % limit	—
`upper`	out	upper 95 % limit	—

Applies when: |phi| < 1; known parameters; normal innovations for the interval.

```
$ time-series solve ar1_forecast x=58 phi=0.7 mu=50 h=3
forecast = 52.744
  by: AR(1) forecast towards the mean  (closed forms of the AR(1) recursion)
  x = 58
  phi = 0.7
  mu = 50
  assumes: |phi| < 1; known parameters; normal innovations for the interval
```

In the worked example, x = 58, phi = 0.7, mu = 50, h = 3 go through the equation above and give forecast = 52.744. It applies only when |phi| < 1; known parameters; normal innovations for the interval: outside that it says what is wrong instead of returning a number.

AR(p) estimation by conditional least squares — `ar_least_squares`

Fit x_t = c + phi_1 x_{t-1} + ... + phi_p x_{t-p} + e_t by ordinary least squares on t = p+1..n; the innovation variance is the residual sum of squares divided by n - p.

$$\min_{c,\phi} \sum_{t=p+1}^{n} \left(x_t - c - \sum_{j=1}^{p} \phi_j x_{t-j} \right)^2$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`p`	in	the AR order p	—
`const`	out	the constant c	—
`phi`	out	the coefficients phi_1..phi_p	—
`phi1`	out	phi_1	—
`sigma2`	out	residual sum of squares / (n - p)	—

Applies when: n - p > p + 2; regressors not collinear; the first p values are conditioned on.

```
$ time-series solve ar_least_squares "x=[4, 6, 5, 8, 7, 9, 8, 10, 9, 12]" p=1
const = 3.5625
phi = [0.6354166666666665]
phi1 = 0.6354166667
sigma2 = 2.515046296
  by: AR(p) estimation by conditional least squares  (least squares (Householder QR) of x_t on (1, x_{t-1}, ..., x_{t-p}), t =
p+1..n)
  assumes: n - p > p + 2; regressors not collinear; the first p values are conditioned on
```

What the worked example means: This procedure fits an autoregressive model of order p by minimizing the sum of squared errors for observations after the initial p values. The resulting constant term is 3.5625, while the lag coefficients are given in [0.635417]. The innovation variance, calculated as the residual sum of squares divided by the degrees of freedom, is 2.51505. This approach conditions on the first p data points, treating them as fixed starting values rather than estimating their distribution.

AR(p) estimation by Yule-Walker — `ar_yule_walker`

Estimate the coefficients phi_1..phi_p and the innovation variance of an AR(p) model from data by the Yule-Walker equations (sample autocovariances divided by n, mean removed).

$$\begin{pmatrix} 1 & r_1 & \cdots \\ r_1 & 1 & \cdots \\ \vdots & & \ddots \end{pmatrix} \phi = \begin{pmatrix} r_1 \\ \vdots \\ r_p \end{pmatrix}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`p`	in	the AR order p	—
`phi`	out	the coefficients phi_1..phi_p	—
`phi1`	out	phi_1	—
`sigma2`	out	the innovation variance c_0 (1 - sum phi_j r_j)	—

Applies when: the series is not constant; 1 <= p < n - 1; denominator n.

```
$ time-series solve ar_yule_walker "x=[4, 6, 5, 8, 7, 9, 8, 10, 9, 12]" p=2
phi = [0.21521751734534134,0.40165819531144314]
phi1 = 0.2152175173
sigma2 = 3.767657229
  by: AR(p) estimation by Yule-Walker  (solve the Toeplitz system R phi = r (Durbin-Levinson); sigma^2 = c_0 prod(1 - phi_kk^2))
  assumes: the series is not constant; 1 <= p < n - 1; denominator n
```

What the worked example means: This calculation estimates the autoregressive parameters for order 2 using the Yule-Walker equations. It solves a system of linear equations based on sample autocovariances to determine the coefficients [0.215218, 0.401658], specifically identifying phi one as 0.215218. The resulting innovation variance, which measures the unpredictability of the series given past values, is computed as 3.76766. These values describe how much current observations depend on previous ones in the stationary process.

Forecasts of an AR(p) model — `ar_forecast`

h-step forecasts of x_t = c + phi_1 x_{t-1} + ... + phi_p x_{t-p} + e_t from the last p observations (future shocks set to 0).

$$\hat{x}_{n+j} = c + \sum_{i=1}^{p} \phi_i \, \hat{x}_{n+j-i}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (at least p values)	—
`phi`	in	the coefficients phi_1..phi_p	—
`c`	in	the constant c (default 0) (optional)	—
`h`	in	forecast horizon (default 1) (optional)	—
`forecasts`	out	the forecasts for steps 1..h	—
`forecast`	out	the h-step forecast	—

Applies when: phi_1 multiplies the latest observation.

```
$ time-series solve ar_forecast "phi=[0.5, 0.2]" "x=[10, 11, 12]" c=1 h=3
forecasts = [9.2,8.0,6.84]
forecast = 6.84
  by: Forecasts of an AR(p) model  (recursion xhat_{n+j} = c + sum_i phi_i xhat_{n+j-i} (observed values where available))
  c = 1
  assumes: phi_1 multiplies the latest observation
```

What the worked example means: This calculation produces forecasts for an Autoregressive model of order p using the recursion formula. It combines the constant term with weighted past values, where future shocks are set to zero. The result shows the predicted value at horizon 3, which is 6.84, based on the series ending in [10, 11, 12] and coefficients [0.5, 0.2].

Forecasts of an ARIMA(p,1,0) model — `arima_p10_forecast`

h-step forecasts of an ARIMA(p,1,0) model without constant: the differences follow a stationary AR(p); forecast differences are accumulated onto the last observation.

$$(1 - \phi_1 L - \cdots - \phi_p L^p)(1 - L) x_t = e_t$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (at least p + 2 values)	—
`phi`	in	AR coefficients of the differences	—
`h`	in	forecast horizon (default 1) (optional)	—
`forecasts`	out	the forecasts for steps 1..h	—
`forecast`	out	the h-step forecast	—

Applies when: stationary AR part; no constant (no drift); n >= p + 2.

```
$ time-series solve arima_p10_forecast "phi=[0.4]" "x=[100, 102, 105, 104, 108]" h=3
forecasts = [109.6,110.24,110.496]
forecast = 110.496
  by: Forecasts of an ARIMA(p,1,0) model  (forecast d_t = x_t - x_{t-1} by the AR(p) recursion, xhat_{n+j} = xhat_{n+j-1} +
dhat_{n+j})
  assumes: stationary AR part; no constant (no drift); n >= p + 2
```

What the worked example means: This calculation produces an h-step forecast for an ARIMA model with a single integration order and no drift. It begins by modeling the differenced series as a stationary autoregressive process of order p, using coefficients [0.4] to predict future changes in the level. These predicted differences are then summed cumulatively starting from the most recent observed value [100, 102, 105, 104, 108]. The resulting output 110.496 represents the expected level of the series at horizon 3, assuming the underlying trend continues without additional constant shifts.

Partial autocorrelations from autocorrelations — `partial_autocorrelations_from_autocorrelations`

The partial autocorrelations phi_11..phi_pp implied by autocorrelations rho_1..rho_p (Durbin-Levinson).

$$\phi_{kk} = \frac{\rho_k - \sum_{j=1}^{k-1} \phi_{k-1,j}\rho_{k-j}}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j}\rho_j}$$

symbol		meaning	SI unit
`rho`	in	the autocorrelations rho_1..rho_p	—
`pacf`	out	partial autocorrelations at lags 1..p	—
`last`	out	phi_pp	—

Applies when: the autocorrelations form a positive definite sequence.

```
$ time-series solve partial_autocorrelations_from_autocorrelations "rho=[0.6, 0.3, 0.1]"
pacf = [0.6,-0.09375,-0.06403940886699507]
last = -0.06403940887
  by: Partial autocorrelations from autocorrelations  (Durbin-Levinson recursion)
  assumes: the autocorrelations form a positive definite sequence
```

What the worked example means: This calculation determines the partial autocorrelations implied by a given sequence of autocorrelations using the Durbin-Levinson recursion. The process iteratively isolates the direct relationship between observations separated by specific lags, removing the influence of intermediate time points. The resulting value for the last lag, -0.0640394, represents the strength of this direct link after accounting for all prior dependencies captured in the output set [0.6, -0.09375, -0.0640394].

Yule-Walker equations from autocorrelations — `yule_walker_from_autocorrelations`

Solve the Yule-Walker equations for the AR(p) coefficients given the autocorrelations rho_1..rho_p, and the ratio of the innovation variance to the process variance.

$$\rho_k = \sum_{j=1}^{p} \phi_j \, \rho_{|k-j|}, \; k = 1..p$$

symbol		meaning	SI unit
`rho`	in	the autocorrelations rho_1..rho_p	—
`phi`	out	the AR coefficients phi_1..phi_p	—
`phi1`	out	phi_1	—
`variance_ratio`	out	sigma^2 / gamma_0 = 1 - sum phi_j rho_j	—

Applies when: the autocorrelations form a positive definite sequence.

```
$ time-series solve yule_walker_from_autocorrelations "rho=[0.6, 0.2]"
phi = [0.75,-0.24999999999999994]
phi1 = 0.75
variance_ratio = 0.6
  by: Yule-Walker equations from autocorrelations  (Durbin-Levinson recursion on rho_1..rho_p)
  assumes: the autocorrelations form a positive definite sequence
```

What the worked example means: This calculation determines the parameters for an autoregressive model of order p by applying the Durbin-Levinson recursion to the provided autocorrelations. The resulting coefficients, denoted as [0.75, -0.25], define how past values influence the current value in the process. Additionally, the method computes the ratio of the innovation variance to the total process variance, shown here as 0.6, which indicates the proportion of variability that cannot be explained by the linear dependence on previous observations.

ARMA theory (13)

h-step forecast error variance of an ARMA model — `forecast_error_variance`

The variance (and standard deviation) of the h-step-ahead forecast error of a stationary ARMA(p,q) with known parameters: sigma^2 (psi_0^2 + ... + psi_{h-1}^2).

$$\operatorname{Var}(e_n(h)) = \sigma^2 \sum_{j=0}^{h-1} \psi_j^2$$

symbol		meaning	SI unit
`phi`	in	AR coefficients ([0] for none)	—
`theta`	in	MA coefficients ([0] for none)	—
`sigma2`	in	innovation variance sigma^2	—
`h`	in	forecast horizon	—
`variance`	out	forecast error variance	—
`sd`	out	forecast error standard deviation	—

Applies when: stationary AR part; known parameters (no estimation error).

```
$ time-series solve forecast_error_variance "phi=[0.5]" "theta=[0.3]" sigma2=2 h=3
variance = 3.6
sd = 1.897366596
  by: h-step forecast error variance of an ARMA model  (psi weights, then sigma^2 sum_{j=0}^{h-1} psi_j^2)
  sigma2 = 2
  assumes: stationary AR part; known parameters (no estimation error)
```

What the worked example means: This calculation determines the uncertainty of a forecast made h steps into the future using an ARMA model. It computes the variance of the forecast error as 3.6, derived from the innovation variance and the sum of squared psi weights up to horizon h-1. The corresponding standard deviation, which represents the typical magnitude of the error, is 1.89737. This result assumes that the autoregressive part of the model is stationary and that all parameters are known exactly.

Invertibility of an MA(2) process — `ma2_invertibility`

Whether x_t = e_t + theta1 e_{t-1} + theta2 e_{t-2} is invertible: the largest modulus of the inverse roots (roots of z^2 + theta1 z + theta2) must be below 1.

$$z^2 + \theta_1 z + \theta_2 = 0, \quad \text{invertible} \iff \max|z_i| < 1$$

symbol		meaning	SI unit
`theta1`	in	theta_1	—
`theta2`	in	theta_2	—
`invertible`	out	yes or no	—
`max_modulus`	out	largest modulus of the inverse roots	—
`roots`	out	real or complex	—

Applies when: plus-sign MA convention; a modulus of exactly 1 counts as not invertible.

```
$ time-series solve ma2_invertibility theta1=0.5 theta2=0.3
invertible = yes
max_modulus = 0.5477225575
roots = complex
  by: Invertibility of an MA(2) process  (roots of z^2 + theta1 z + theta2 by the quadratic formula)
  theta1 = 0.5
  theta2 = 0.3
  assumes: plus-sign MA convention; a modulus of exactly 1 counts as not invertible
```

What the worked example means: This calculation determines if the moving average process is invertible by finding the roots of the characteristic equation using the provided coefficients. The result shows that the largest modulus of these inverse roots is 0.547723, leading to an assessment of yes. Invertibility ensures that the current error term can be uniquely recovered from past observations, which is crucial for accurate forecasting and parameter estimation.

Mean, variance and autocorrelation of an AR(1) process — `ar1_properties`

For a stationary AR(1) x_t = c + phi x_{t-1} + e_t: the mean c/(1 - phi), the variance sigma^2/(1 - phi^2), and the lag-k autocorrelation phi^k and autocovariance.

$$\mu = \frac{c}{1-\phi}, \ \gamma_0 = \frac{\sigma^2}{1-\phi^2}, \ \rho_k = \phi^k$$

symbol		meaning	SI unit
`phi`	in	AR coefficient, |phi| < 1	—
`sigma2`	in	innovation variance sigma^2	—
`k`	in	the lag k (default 1) (optional)	—
`c`	in	the constant c (default 0) (optional)	—
`mean`	out	c / (1 - phi)	—
`variance`	out	sigma^2 / (1 - phi^2)	—
`acf_k`	out	phi^k	—
`acov_k`	out	variance x phi^k	—

Applies when: |phi| < 1; sigma^2 > 0.

```
$ time-series solve ar1_properties phi=0.6 sigma2=4 c=2
mean = 5
variance = 6.25
acf_k = 0.6
acov_k = 3.75
  by: Mean, variance and autocorrelation of an AR(1) process  (stationary moments of the AR(1) recursion)
  phi = 0.6
  sigma2 = 4
  c = 2
  assumes: |phi| < 1; sigma^2 > 0
```

In the worked example, phi = 0.6, sigma2 = 4, c = 2 go through the equation above and give mean = 5, variance = 6.25, acf_k = 0.6, acov_k = 3.75. It applies only when |phi| < 1; sigma^2 > 0: outside that it says what is wrong instead of returning a number.

Pi weights (AR-infinity representation) of an ARMA process — `pi_weights`

The weights pi_1..pi_K of x_t = sum_{j>=1} pi_j x_{t-j} + e_t for an invertible ARMA(p,q).

$$\pi(L) = 1 - \sum_{j \geq 1} \pi_j L^j = \frac{\phi(L)}{\theta(L)}$$

symbol		meaning	SI unit
`phi`	in	AR coefficients ([0] for none)	—
`theta`	in	MA coefficients ([0] for none), invertible	—
`K`	in	number of weights (default 10) (optional)	—
`pi`	out	pi_1..pi_K	—
`pi1`	out	pi_1 = phi_1 + theta_1	—

Applies when: invertible MA part; stationary AR part; plus-sign MA convention.

```
$ time-series solve pi_weights "phi=[0.5]" "theta=[0.3]" K=5
pi = [0.8,-0.24,0.072,-0.021599999999999998,0.006479999999999999]
pi1 = 0.8
  by: Pi weights (AR-infinity representation) of an ARMA process  (coefficients c_j of phi(L)/theta(L); pi_j = -c_j)
  assumes: invertible MA part; stationary AR part; plus-sign MA convention
```

What the worked example means: This calculation determines the autoregressive weights for an invertible ARMA process, expressing it as an infinite sum of past values. The primary weight, 0.8, captures the direct effect of the lag-one observation on the current value, derived from the input coefficients [0.5] and [0.3]. These weights allow practitioners to interpret how historical data points contribute to the present state under stationarity assumptions.

Psi weights (MA-infinity representation) of an ARMA process — `psi_weights`

The weights psi_1..psi_K of x_t = sum_{j>=0} psi_j e_{t-j} (psi_0 = 1) for a stationary ARMA(p,q); they are also the impulse response.

$$\psi_j = \theta_j + \sum_{i=1}^{\min(j,p)} \phi_i \, \psi_{j-i}, \; \psi_0 = 1$$

symbol		meaning	SI unit
`phi`	in	AR coefficients ([0] for none)	—
`theta`	in	MA coefficients ([0] for none)	—
`K`	in	number of weights (default 10) (optional)	—
`psi`	out	psi_1..psi_K	—
`psi1`	out	psi_1 = phi_1 + theta_1	—

Applies when: stationary AR part; plus-sign MA convention.

```
$ time-series solve psi_weights "phi=[0.5]" "theta=[0.3]" K=5
psi = [0.8,0.4,0.2,0.1,0.05]
psi1 = 0.8
  by: Psi weights (MA-infinity representation) of an ARMA process  (psi_j = theta_j + sum_{i=1}^{min(j,p)} phi_i psi_{j-i})
  assumes: stationary AR part; plus-sign MA convention
```

What the worked example means: This calculation determines the impulse response weights for an autoregressive moving average process. The first weight, 0.8, equals the sum of the first autoregressive coefficient and the first moving average coefficient. These values describe how past errors influence the current observation, with the magnitude indicating the persistence of shocks in the time series.

Stationarity of an AR(2) process — `ar2_stationarity`

Whether x_t = phi1 x_{t-1} + phi2 x_{t-2} + e_t is stationary: the largest modulus of the inverse characteristic roots (roots of z^2 - phi1 z - phi2) must be below 1.

$$z^2 - \phi_1 z - \phi_2 = 0, \quad \text{stationary} \iff \max |z_i| < 1$$

symbol		meaning	SI unit
`phi1`	in	phi_1	—
`phi2`	in	phi_2	—
`stationary`	out	yes or no	—
`max_modulus`	out	largest modulus of the inverse roots	—
`roots`	out	real or complex	—

Applies when: a modulus of exactly 1 (unit root) counts as not stationary.

```
$ time-series solve ar2_stationarity phi1=0.5 phi2=0.3
stationary = yes
max_modulus = 0.8520797289
roots = real
  by: Stationarity of an AR(2) process  (roots of z^2 - phi1 z - phi2 by the quadratic formula)
  phi1 = 0.5
  phi2 = 0.3
  assumes: a modulus of exactly 1 (unit root) counts as not stationary
```

What the worked example means: This calculation determines if the autoregressive model is stable by solving for the characteristic roots using the coefficients 0.5 and 0.3. The result shows that the largest modulus among these roots is 0.85208, which indicates the process is yes. For stability, this value must be strictly less than one; otherwise, the series may exhibit explosive behavior or contain a unit root.

Theoretical autocorrelations of an AR(2) process — `ar2_theoretical_acf`

The autocorrelations rho_1..rho_K of a stationary AR(2) process from the Yule-Walker recursion rho_1 = phi1/(1 - phi2), rho_k = phi1 rho_{k-1} + phi2 rho_{k-2}.

$$\rho_1 = \frac{\phi_1}{1-\phi_2},\ \rho_k = \phi_1\rho_{k-1} + \phi_2\rho_{k-2}$$

symbol		meaning	SI unit
`phi1`	in	phi_1	—
`phi2`	in	phi_2	—
`K`	in	number of lags (default 5) (optional)	—
`acf`	out	rho_1..rho_K	—
`rho1`	out	rho_1	—

Applies when: stationary AR(2).

```
$ time-series solve ar2_theoretical_acf phi1=0.5 phi2=0.3
acf = [0.7142857142857143,0.6571428571428571,0.5428571428571428,0.4685714285714285,0.3971428571428713]
rho1 = 0.7142857143
  by: Theoretical autocorrelations of an AR(2) process  (Yule-Walker recursion)
  phi1 = 0.5
  phi2 = 0.3
  assumes: stationary AR(2)
```

What the worked example means: This calculation determines the theoretical autocorrelations for a stationary autoregressive process of order two. Using the Yule-Walker recursion, it computes the correlation at lag one as 0.714286 based on the coefficients 0.5 and 0.3. The resulting series [0.714286, 0.657143, 0.542857, 0.468571, 0.397143] describes how observations relate to their past values across multiple lags.

Theoretical autocorrelations of an ARMA(1,1) process — `arma11_theoretical_acf`

The autocorrelations rho_1..rho_K of x_t = phi x_{t-1} + e_t + theta e_{t-1}.

$$\rho_1 = \frac{(1+\phi\theta)(\phi+\theta)}{1+2\phi\theta+\theta^2},\ \rho_k = \phi\,\rho_{k-1}$$

symbol		meaning	SI unit
`phi`	in	AR coefficient, |phi| < 1	—
`theta`	in	MA coefficient (plus-sign convention)	—
`K`	in	number of lags (default 5) (optional)	—
`acf`	out	rho_1..rho_K	—
`rho1`	out	rho_1	—

Applies when: |phi| < 1; x_t = phi x_{t-1} + e_t + theta e_{t-1}.

```
$ time-series solve arma11_theoretical_acf phi=0.6 theta=0.3
acf = [0.7324137931034482,0.4394482758620689,0.26366896551724134,0.1582013793103448,0.09492082758620689]
rho1 = 0.7324137931
  by: Theoretical autocorrelations of an ARMA(1,1) process  (rho_1 = (1 + phi theta)(phi + theta)/(1 + 2 phi theta + theta^2);
rho_k = phi rho_{k-1})
  phi = 0.6
  theta = 0.3
  assumes: |phi| < 1; x_t = phi x_{t-1} + e_t + theta e_{t-1}
```

What the worked example means: This calculation determines the theoretical autocorrelations for an ARMA(1,1) process using the specified autoregressive and moving average coefficients. The lag-one correlation is computed as 0.732414, while subsequent values follow a geometric decay pattern defined by the autoregressive parameter. These results represent the expected linear dependence between observations separated by specific time intervals under the assumption of stationarity.

Theoretical autocorrelations of an ARMA(p,q) process — `arma_theoretical_acf`

The autocorrelations rho_1..rho_K of a stationary ARMA(p,q) process x_t = sum phi_i x_{t-i} + e_t + sum theta_j e_{t-j}.

$$\gamma_k - \sum_{i=1}^{p} \phi_i \gamma_{k-i} = \sigma^2 \sum_{j=k}^{q} \theta_j \psi_{j-k}$$

symbol		meaning	SI unit
`phi`	in	AR coefficients phi_1..phi_p (stationary; [0] for none)	—
`theta`	in	MA coefficients theta_1..theta_q ([0] for none)	—
`K`	in	number of lags (default 5) (optional)	—
`acf`	out	rho_1..rho_K	—
`rho1`	out	rho_1	—

Applies when: stationary AR part; plus-sign MA convention.

```
$ time-series solve arma_theoretical_acf "phi=[0.5, 0.2]" "theta=[0.4]"
acf = [0.8012048192771084,0.6006024096385542,0.46054216867469877,0.35039156626506024,0.2673042168674699]
rho1 = 0.8012048193
  by: Theoretical autocorrelations of an ARMA(p,q) process  (solve the linear equations for gamma_0..gamma_p, then gamma_k = sum
phi_i gamma_{k-i} + sigma^2 sum theta_j psi_{j-k})
  assumes: stationary AR part; plus-sign MA convention
```

What the worked example means: The theoretical autocorrelation at lag one is 0.801205, derived from the AR coefficients [0.5, 0.2] and MA coefficients [0.4].

Theoretical autocorrelations of an MA(q) process — `ma_theoretical_acf`

The autocorrelations rho_1..rho_K of x_t = e_t + theta_1 e_{t-1} + ... + theta_q e_{t-q}; they vanish beyond lag q.

$$\rho_k = \frac{\theta_k + \sum_{j=1}^{q-k} \theta_j \theta_{j+k}}{1 + \sum_{j=1}^{q} \theta_j^2}$$

symbol		meaning	SI unit
`theta`	in	the MA coefficients theta_1..theta_q (plus-sign convention)	—
`K`	in	number of lags (default q + 1) (optional)	—
`acf`	out	rho_1..rho_K	—
`rho1`	out	rho_1	—

Applies when: x_t = e_t + sum theta_j e_{t-j} (plus signs).

```
$ time-series solve ma_theoretical_acf "theta=[0.5, 0.3]"
acf = [0.48507462686567165,0.2238805970149253,0.0]
rho1 = 0.4850746269
  by: Theoretical autocorrelations of an MA(q) process  (rho_k = (theta_k + sum_{j=1}^{q-k} theta_j theta_{j+k}) / (1 + sum
theta_j^2), 0 for k > q)
  assumes: x_t = e_t + sum theta_j e_{t-j} (plus signs)
```

In the worked example, theta = [0.5, 0.3] go through the equation above and give acf = [0.4850746269, 0.223880597, 0], rho1 = 0.4850746269. It applies only when x_t = e_t + sum theta_j e_{t-j} (plus signs): outside that it says what is wrong instead of returning a number.

Theoretical partial autocorrelations of an ARMA(p,q) process — `arma_theoretical_pacf`

The partial autocorrelations phi_11..phi_KK of a stationary ARMA(p,q) process (Durbin-Levinson on its exact autocorrelations).

$$\phi_{kk} = \frac{\rho_k - \sum_{j=1}^{k-1} \phi_{k-1,j} \rho_{k-j}}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j} \rho_j}$$

symbol		meaning	SI unit
`phi`	in	AR coefficients ([0] for none)	—
`theta`	in	MA coefficients ([0] for none)	—
`K`	in	number of lags (default 5) (optional)	—
`pacf`	out	phi_11..phi_KK	—
`pacf1`	out	phi_11 = rho_1	—

Applies when: stationary AR part; plus-sign MA convention.

```
$ time-series solve arma_theoretical_pacf "phi=[0.5]" "theta=[0.4]"
pacf = [0.6923076923076924,-0.2556818181818184,0.1010327795240235,-0.04033486904612524,0.016128951324258605]
pacf1 = 0.6923076923
  by: Theoretical partial autocorrelations of an ARMA(p,q) process  (exact autocovariances, then the Durbin-Levinson recursion)
  assumes: stationary AR part; plus-sign MA convention
```

What the worked example means: This calculation determines the theoretical partial autocorrelations for a stationary Autoregressive Moving Average process using the Durbin-Levinson recursion on exact autocovariances. The result, represented as [0.692308, -0.255682, 0.101033, -0.0403349, 0.016129], quantifies the direct linear relationship between observations separated by specific lags after accounting for intermediate correlations. For instance, the first partial autocorrelation 0.692308 equals the lag-one autocorrelation, while subsequent values depend on the AR coefficients [0.5] and MA coefficients [0.4]. These values help identify the order of the autoregressive component in the time series model.

Theoretical partial autocorrelations of an MA(1) process — `ma1_theoretical_pacf`

The partial autocorrelations phi_11..phi_KK of x_t = e_t + theta e_{t-1} (they decay, never cut off).

$$\phi_{kk} = -\frac{(-\theta)^k(1-\theta^2)}{1-\theta^{2(k+1)}}$$

symbol		meaning	SI unit
`theta`	in	MA coefficient, |theta| != 1	—
`K`	in	number of lags (default 5) (optional)	—
`pacf`	out	phi_11..phi_KK	—
`pacf1`	out	phi_11 = theta/(1 + theta^2)	—

Applies when: |theta| != 1; plus-sign MA convention.

```
$ time-series solve ma1_theoretical_pacf theta=0.6
pacf = [0.441176470588,-0.241675617615,0.140601566406,-0.0834485816626,0.0498749669471]
pacf1 = 0.4411764706
  by: Theoretical partial autocorrelations of an MA(1) process  (closed form phi_kk = -(-theta)^k (1 - theta^2)/(1 -
theta^(2(k+1))))
  theta = 0.6
  assumes: |theta| != 1; plus-sign MA convention
```

What the worked example means: This calculation determines the theoretical partial autocorrelations for an MA(1) process using the coefficient 0.6. The result, represented by [0.441176, -0.241676, 0.140602, -0.0834486, 0.049875], illustrates how these values decline over lags rather than cutting off abruptly. Specifically, the first value 0.441176 establishes the starting point of this decay pattern.

Variance and lag-1 autocovariance of an ARMA(p,q) process — `arma_variance`

The process variance gamma_0 and lag-1 autocovariance gamma_1 of a stationary ARMA(p,q) process with innovation variance sigma^2.

$$\gamma_0 = \sigma^2 \sum_{j \geq 0} \psi_j^2$$

symbol		meaning	SI unit
`phi`	in	AR coefficients ([0] for none)	—
`theta`	in	MA coefficients ([0] for none)	—
`sigma2`	in	innovation (white-noise) variance sigma^2	—
`gamma0`	out	the variance of the process	—
`gamma1`	out	the lag-1 autocovariance	—

Applies when: stationary AR part; sigma^2 > 0.

```
$ time-series solve arma_variance "phi=[0.5]" "theta=[0.4]" sigma2=2
gamma0 = 4.16
gamma1 = 2.88
  by: Variance and lag-1 autocovariance of an ARMA(p,q) process  (exact linear equations for the autocovariances)
  sigma2 = 2
  assumes: stationary AR part; sigma^2 > 0
```

What the worked example means: This calculation determines the long-run variability and immediate dependence of a stationary autoregressive moving average process. Given the autoregressive coefficients, moving average coefficients, and innovation variance, the method solves exact linear equations for the autocovariances. The resulting variance, 4.16, indicates the overall scale of fluctuations in the series, while the lag-one autocovariance, 2.88, quantifies how strongly consecutive observations are related.

Autocorrelation (6)

Bartlett standard error of a sample autocorrelation — `bartlett_standard_error`

Bartlett's standard error of r_k under the hypothesis that the autocorrelations beyond lag k - 1 are zero, and the 95 % band half-width used in correlograms.

$$\mathrm{se}(r_k) = \sqrt{\frac{1 + 2\sum_{j=1}^{k-1} r_j^2}{n}}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`k`	in	the lag k	—
`se`	out	Bartlett standard error of r_k	—
`half_width_95`	out	1.959964 x se	—
`acf`	out	the sample autocorrelation r_k	—

Applies when: 1 <= k <= n - 1; autocorrelations with denominator n.

```
$ time-series solve bartlett_standard_error "x=[4, 6, 5, 8, 7, 9, 8, 10, 9, 12]" k=3
se = 0.4144598129
half_width_95 = 0.8123263063
acf = -0.04108527132
  by: Bartlett standard error of a sample autocorrelation  (se_k = sqrt((1 + 2 sum_{j<k} r_j^2) / n))
  assumes: 1 <= k <= n - 1; autocorrelations with denominator n
```

What the worked example means: This calculation determines the Bartlett standard error for the sample autocorrelation at lag 3. It uses the sum of squared autocorrelations from previous lags and the sample size to estimate uncertainty under the assumption that correlations beyond lag 3 minus one are zero. The resulting standard error is 0.41446, which translates to a confidence band half-width of 0.812326. This value helps assess whether the observed autocorrelation of -0.0410853 is statistically significant.

Box-Pierce test — `box_pierce_test`

The Box-Pierce test portmanteau statistic for no autocorrelation up to lag h and its chi-square p-value (h - model_df degrees of freedom; autocorrelations with denominator n).

$$Q_{BP} = n\sum_{k=1}^{h} r_k^2$$

symbol		meaning	SI unit
`x`	in	the series (or residuals), oldest first	—
`h`	in	the number of lags h	—
`model_df`	in	parameters of a fitted ARMA model subtracted from the degrees of freedom (default 0) (optional)	—
`q`	out	the test statistic	—
`p_value`	out	upper-tail chi-square probability, df = h - model_df	—
`df`	out	degrees of freedom h - model_df	—

Applies when: 1 <= h <= n - 1; model_df < h (default 0: raw series).

```
$ time-series solve box_pierce_test "x=[4, 6, 5, 8, 7, 9, 8, 10, 9, 12, 11, 13]" h=3
q = 6.943896066
p_value = 0.07370765771
df = 3
  by: Box-Pierce test  (Q = n sum_{k=1}^h r_k^2; p = P(chi2_{h-model_df} > Q))
  assumes: 1 <= h <= n - 1; model_df < h (default 0: raw series)
```

What the worked example means: This calculation determines the Box-Pierce portmanteau statistic to assess whether significant autocorrelation exists in the series up to lag 3. The resulting statistic is 6.9439, derived from the sum of squared sample autocorrelations scaled by the sample size. With 3 degrees of freedom, the associated p-value is 0.0737077, which indicates the probability of observing such a statistic under the null hypothesis of no serial dependence.

Ljung-Box test — `ljung_box_test`

The Ljung-Box test portmanteau statistic for no autocorrelation up to lag h and its chi-square p-value (h - model_df degrees of freedom; autocorrelations with denominator n).

$$Q_{LB} = n(n+2)\sum_{k=1}^{h} \frac{r_k^2}{n-k}$$

symbol		meaning	SI unit
`x`	in	the series (or residuals), oldest first	—
`h`	in	the number of lags h	—
`model_df`	in	parameters of a fitted ARMA model subtracted from the degrees of freedom (default 0) (optional)	—

symbol		meaning	SI unit
`q`	out	the test statistic	—
`p_value`	out	upper-tail chi-square probability, df = h - model_df	—
`df`	out	degrees of freedom h - model_df	—

Applies when: 1 <= h <= n - 1; model_df < h (default 0: raw series).

```
$ time-series solve ljung_box_test "x=[4, 6, 5, 8, 7, 9, 8, 10, 9, 12, 11, 13]" h=3
q = 9.319101598
p_value = 0.0253357685
df = 3
  by: Ljung-Box test  (Q = n(n+2) sum_{k=1}^h r_k^2/(n-k); p = P(chi2_{h-model_df} > Q))
  assumes: 1 <= h <= n - 1; model_df < h (default 0: raw series)
```

What the worked example means: This calculation determines the Ljung-Box portmanteau test statistic for autocorrelation up to lag 3 using the series or residuals provided. The resulting statistic 9.3191 measures the cumulative deviation of sample autocorrelations from zero, adjusted for sample size and model complexity. With 3 degrees of freedom derived from the number of lags minus the model parameters, the associated p-value 0.0253358 indicates whether the observed autocorrelation structure is statistically significant.

Sample autocorrelation at lag k — `sample_autocorrelation`

The sample autocorrelation r_k of a series (and r_1..r_k), with autocovariances divided by n and taken about the sample mean.

$$r_k = \frac{\sum_{t=1}^{n-k}(x_t - \bar{x})(x_{t+k} - \bar{x})}{\sum_{t=1}^{n}(x_t - \bar{x})^2}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`k`	in	the lag k (1 <= k <= n - 1)	—
`acf`	out	the sample autocorrelation r_k	—
`acf_list`	out	r_1, ..., r_k	—

Applies when: the series is not constant; 1 <= k <= n - 1; denominator n (not n - k).

```
$ time-series solve sample_autocorrelation "x=[4, 6, 5, 8, 7, 9, 8, 10]" k=1
acf = 0.2721861472
acf_list = [0.2721861471861472]
  by: Sample autocorrelation at lag k  (r_k = c_k / c_0, c_k = (1/n) sum_{t=1}^{n-k} (x_t - xbar)(x_{t+k} - xbar))
  assumes: the series is not constant; 1 <= k <= n - 1; denominator n (not n - k)
```

What the worked example means: This calculation determines the linear relationship between data points separated by 1 time steps in the series [4, 6, 5, 8, 7, 9, 8, 10]. The result, 0.272186, represents the correlation coefficient for that specific lag. A value near one indicates strong positive repetition, while a value near zero suggests no linear connection between observations at that distance.

Sample autocovariance at lag k — `sample_autocovariance`

The sample autocovariance c_k of a series, divided by n and taken about the sample mean.

$$c_k = \frac{1}{n}\sum_{t=1}^{n-k}(x_t - \bar{x})(x_{t+k} - \bar{x})$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`k`	in	the lag k (0 <= k <= n - 1)	—
`acov`	out	the sample autocovariance c_k	—
`gamma0`	out	c_0, the variance divided by n	—

Applies when: 0 <= k <= n - 1; denominator n (not n - k).

```
$ time-series solve sample_autocovariance "x=[4, 6, 5, 8, 7, 9, 8, 10]" k=2
acov = 1.60546875
gamma0 = 3.609375
  by: Sample autocovariance at lag k  (c_k = (1/n) sum_{t=1}^{n-k} (x_t - xbar)(x_{t+k} - xbar))
  assumes: 0 <= k <= n - 1; denominator n (not n - k)
```

What the worked example means: This calculation determines how strongly values in the series relate to each other when separated by a specific time gap. It uses the mean of the entire dataset to center the data before multiplying pairs of observations that are k steps apart, then averaging these products over the total number of observations n. The resulting value, 1.60547, quantifies this linear dependence, while 3.60938 represents the total variability of the series around its mean.

Sample partial autocorrelation at lag k — `sample_partial_autocorrelation`

The sample partial autocorrelation phi_kk of a series (and phi_11..phi_kk) by the Durbin-Levinson recursion on the sample autocorrelations (denominator n).

$$\phi_{kk} = \frac{r_k - \sum_{j=1}^{k-1} \phi_{k-1,j}\, r_{k-j}}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j}\, r_j}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`k`	in	the lag k (1 <= k < n/2)	—
`pacf`	out	the partial autocorrelation at lag k	—
`pacf_list`	out	partial autocorrelations at lags 1..k	—

Applies when: the series is not constant; 1 <= k < n/2; autocovariances divided by n.

```
$ time-series solve sample_partial_autocorrelation "x=[4, 6, 5, 8, 7, 9, 8, 10, 9, 12]" k=2
pacf = 0.4016581953
pacf_list = [0.3596899224806201,0.40165819531144314]
  by: Sample partial autocorrelation at lag k  (Durbin-Levinson: phi_kk = (r_k - sum phi_{k-1,j} r_{k-j}) / (1 - sum phi_{k-1,j}
r_j))
  assumes: the series is not constant; 1 <= k < n/2; autocovariances divided by n
```

What the worked example means: This calculation determines the sample partial autocorrelation at lag 2 using the Durbin-Levinson recursion on the series data. The result, 0.401658, represents the correlation between observations separated by 2 steps, with the influence of all shorter lags removed. A value near zero suggests that the relationship at this distance is explained by closer dependencies, while a significant non-zero value indicates a direct link in the time series structure.

Change points (5)

CUSUM estimate of a change point — `cusum_change_point`

The change point estimated by the cumulative sum of deviations from the overall mean, S_k = sum_{t<=k}(x_t - xbar): the k with the largest |S_k|.

$$S_k = \sum_{t=1}^{k} (x_t - \bar{x}), \quad \hat{k} = \arg\max_k |S_k|$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`change_index`	out	k with the largest |S_k| (first segment x_1..x_k)	—
`max_cusum`	out	max |S_k|	—
`mean_before`	out	mean of x_1..x_k	—
`mean_after`	out	mean of x_{k+1}..x_n	—

Applies when: k = 1..n-1; ties: the smallest k.

```
$ time-series solve cusum_change_point "x=[5, 6, 5, 7, 6, 12, 13, 12, 14]"
change_index = 5
max_cusum = 15.44444444
mean_before = 5.8
mean_after = 12.75
  by: CUSUM estimate of a change point  (running sum of deviations from the mean; argmax of its absolute value)
  assumes: k = 1..n-1; ties: the smallest k
```

What the worked example means: This calculation identifies the single most likely location of an abrupt shift in the mean level of the time series. Summing the deviations of each observation from the overall average tracks how far the running total drifts from zero. The index with the largest absolute cumulative sum, reported here as 5, marks the boundary between the first segment and the second. At this split, the mean of the initial data points is 5.8, while the mean of the remaining points is 12.75, highlighting the magnitude of the detected structural break.

Multiple change points with a penalty (PELT) — `penalised_change_points`

The segmentation minimising the within-segment sum of squares plus a penalty per change point (optimal partitioning, as found by PELT): the number and positions of the changes in the mean.

$$\min_{K,\tau} \sum_{i=0}^{K} \mathrm{SS}(\tau_i{+}1{:}\tau_{i+1}) + \beta K$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`penalty`	in	penalty per change point (> 0)	—
`n_changes`	out	number of change points	—

symbol		meaning	SI unit
`change_indices`	out	ends of all segments but the last	—
`cost`	out	within-segment sum of squares of the chosen segmentation	—

Applies when: penalty > 0; segments of any length >= 1; ties: the earlier last change.

```
$ time-series solve penalised_change_points "x=[5, 6, 5, 12, 13, 12, 20, 21, 19, 20]" penalty=10
n_changes = 2
change_indices = [3,6]
cost = 3.333333333
  by: Multiple change points with a penalty (PELT)  (optimal partitioning F(b) = min_a F(a) + SS(a+1:b) + penalty)
  penalty = 10
  assumes: penalty > 0; segments of any length >= 1; ties: the earlier last change
```

What the worked example means: This method segments the time series by minimizing the within-segment sum of squares plus a penalty for each detected change. The result indicates that the optimal partitioning found 2 distinct shifts in the mean level of the data. These changes occur at the indices listed in [3, 6], and the overall fit quality is quantified by the cost value of 3.33333. This approach balances the accuracy of the segmentation against the complexity of having too many breakpoints.

Single change point in the mean (least squares) — `change_point_mean_shift`

The single change point that minimises the total within-segment sum of squares (a shift in the mean), with the means before and after.

$$\hat{k} = \arg\min_{k} \left[\sum_{t \le k} (x_t - \bar{x}_{1:k})^2 + \sum_{t > k} (x_t - \bar{x}_{k+1:n})^2 \right]$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`min_size`	in	minimum segment length (default 2) (optional)	—
`change_index`	out	k: the first segment is x_1..x_k	—
`mean_before`	out	mean of x_1..x_k	—
`mean_after`	out	mean of x_{k+1}..x_n	—
`cost`	out	total within-segment sum of squares	—
`reduction`	out	sum of squares removed by the split	—

Applies when: each segment has at least min_size values; ties: the smallest k.

```
$ time-series solve change_point_mean_shift "x=[5, 6, 5, 7, 6, 12, 13, 12, 14]"
change_index = 5
mean_before = 5.8
mean_after = 12.75
cost = 5.55
reduction = 107.3388889
  by: Single change point in the mean (least squares)  (exhaustive search over every admissible split)
  assumes: each segment has at least min_size values; ties: the smallest k
```

What the worked example means: The change point at index 5 yields a pre-mean of 5.8 and post-mean of 12.75, reducing the sum of squares by 107.339.

Tabular CUSUM control chart — `tabular_cusum`

The upper and lower tabular CUSUM statistics for a target mean mu0 with allowance k, and the first time either exceeds the decision interval h.

$$C_t^+ = \max(0,\, C_{t-1}^+ + x_t - \mu_0 - k), \quad C_t^- = \max(0,\, C_{t-1}^- + \mu_0 - k - x_t)$$

symbol		meaning	SI unit
`x`	in	the observations, oldest first	—
`mu0`	in	target (in-control) mean	—
`k`	in	allowance (reference value) k >= 0	—
`h`	in	decision interval h > 0	—
`c_plus`	out	final upper CUSUM C+_n	—
`c_minus`	out	final lower CUSUM C-_n	—
`alarm_index`	out	first t with C+_t > h or C-_t > h (0 if none)	—
`side`	out	upper, lower, both or none	—

Applies when: C+_0 = C-_0 = 0; an alarm needs a value strictly above h.

```
$ time-series solve tabular_cusum "x=[10.2, 9.8, 10.5, 11.2, 11.8, 12.1, 11.9]" mu0=10 k=0.5 h=4
c_plus = 5
c_minus = 0
alarm_index = 7
side = upper
  by: Tabular CUSUM control chart  (C+_t = max(0, C+_{t-1} + x_t - mu0 - k); C-_t = max(0, C-_{t-1} + mu0 - k - x_t))
  mu0 = 10
  k = 0.5
  h = 4
  assumes: C+_0 = C-_0 = 0; an alarm needs a value strictly above h
```

What the worked example means: This calculation tracks cumulative deviations from the target mean 10 using an allowance of 0.5. The upper statistic reaches 5 while the lower statistic ends at 0. An alarm is triggered if either value strictly exceeds the decision interval 4, which occurs at observation index 7 for the upper direction, indicating a sustained shift in the process mean.

Two change points in the mean (optimal segmentation) — `two_change_points`

The two change points that minimise the total within-segment sum of squares (three mean levels), found by exhaustive dynamic search.

$$\min_{i<j} \mathrm{SS}(1{:}i) + \mathrm{SS}(i{+}1{:}j) + \mathrm{SS}(j{+}1{:}n)$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`min_size`	in	minimum segment length (default 2) (optional)	—
`change_1`	out	end of the first segment	—
`change_2`	out	end of the second segment	—
`means`	out	the three segment means	—
`cost`	out	total within-segment sum of squares	—

Applies when: each segment has at least min_size values; ties: the smallest first, then second index.

```
$ time-series solve two_change_points "x=[5, 6, 5, 12, 13, 12, 20, 21, 19]"
change_1 = 3
change_2 = 6
means = [5.333333333333333,12.333333333333334,20.0]
cost = 3.333333333
  by: Two change points in the mean (optimal segmentation)  (exhaustive search over all pairs of admissible splits)
  assumes: each segment has at least min_size values; ties: the smallest first, then second index
```

What the worked example means: The optimal segmentation places change points at 3 and 6, yielding a total cost of 3.33333.

Classical decomposition (4)

Seasonal indices (additive decomposition) — `seasonal_indices_additive`

Seasonal indices of classical additive decomposition: detrend by the centred moving average, average each season, normalise the indices to sum to 0.

$$S_i = \bar{d}_i - \frac{1}{m}\sum_j \bar{d}_j, \quad d_t = x_t - \mathrm{CMA}_t$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (at least two full seasons)	—
`m`	in	season length	—
`indices`	out	the m seasonal indices, starting with the season of x_1	—
`first`	out	the index of the first observation's season	—

Applies when: n >= 2m; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i (the list starts at the first observation's season).

```
$ time-series solve seasonal_indices_additive "x=[30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51]" m=4
indices = [6.484375,-6.453125,0.171875,-0.203125]
first = 6.484375
  by: Seasonal indices (additive decomposition)  (differences x_t - CMA_t averaged by season, minus their mean)
  assumes: n >= 2m; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i
(the list starts at the first observation's season)
```

What the worked example means: This calculation determines the fixed seasonal adjustments for an additive time series model. It begins by removing the trend using a centred moving average of length 4 from the input data [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51]. The resulting differences are averaged for each specific season position. These raw averages are then normalized so that their sum is zero, producing the final set of indices [6.48438, -6.45312, 0.171875, -0.203125]. The value 6.48438 represents the adjustment applied to observations occurring in the same season as the very first data point.

Seasonal indices (multiplicative decomposition) — `seasonal_indices_multiplicative`

Seasonal indices of classical multiplicative decomposition: detrend by the centred moving average, average each season, normalise the indices to average 1.

$$S_i = \bar{r}_i \Big/ \tfrac{1}{m}\sum_j \bar{r}_j, \quad r_t = x_t / \mathrm{CMA}_t$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (at least two full seasons)	—
`m`	in	season length	—
`indices`	out	the m seasonal indices, starting with the season of x_1	—
`first`	out	the index of the first observation's season	—

Applies when: n >= 2m; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i (the list starts at the first observation's season); positive data.

```
$ time-series solve seasonal_indices_multiplicative "x=[30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51]" m=4
indices = [1.1675854591936505,0.8390338031104935,1.002044887955251,0.9913358497406046]
first = 1.167585459
  by: Seasonal indices (multiplicative decomposition)  (ratios x_t / CMA_t averaged by season, divided by their mean)
  assumes: n >= 2m; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i
(the list starts at the first observation's season); positive data
```

What the worked example means: This calculation determines the seasonal indices for a multiplicative time series model. It begins by computing the centred moving average of the input series [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51] with season length 4. The ratio of each observation to its corresponding centred moving average is then calculated. These ratios are averaged for each specific season position. Finally, these averages are normalised so that their overall mean equals one, producing the output set [1.16759, 0.839034, 1.00204, 0.991336]. The index for the first observation's season is reported as 1.16759.

Seasonally adjusted series (additive) — `seasonally_adjusted_additive`

The seasonally adjusted series of classical additive decomposition: each value minus its seasonal index.

$$x_t^{\mathrm{SA}} = x_t - S_t$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (at least two full seasons)	—
`m`	in	season length	—
`adjusted`	out	the seasonally adjusted series	—
`last`	out	the latest adjusted value	—

Applies when: n >= 2m; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i (the list starts at the first observation's season).

```
$ time-series solve seasonally_adjusted_additive "x=[30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51]" m=4
adjusted =
[23.515625,27.453125,28.828125,31.203125,33.515625,36.453125,38.828125,41.203125,43.515625,45.453125,47.828125,51.203125]
last = 51.203125
  by: Seasonally adjusted series (additive)  (seasonal indices of classical decomposition, then x_t - S_{season(t)})
  assumes: n >= 2m; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i
(the list starts at the first observation's season)
```

What the worked example means: This calculation applies classical additive decomposition to strip out regular seasonal patterns. Subtracting the specific seasonal index for each time period from the original observation isolates the trend and irregular components. The resulting adjusted series allows for a clearer comparison of underlying performance across different times without calendar-based biases. For instance, the latest adjusted value is 51.2031, which represents the current state after removing expected seasonal fluctuations.

Seasonally adjusted series (multiplicative) — `seasonally_adjusted_multiplicative`

The seasonally adjusted series of classical multiplicative decomposition: each value divided by its seasonal index.

$$x_t^{\mathrm{SA}} = x_t / S_t$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (at least two full seasons)	—
`m`	in	season length	—
`adjusted`	out	the seasonally adjusted series	—
`last`	out	the latest adjusted value	—

Applies when: n >= 2m; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i (the list starts at the first observation's season); positive data.

```
$ time-series solve seasonally_adjusted_multiplicative "x=[30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51]" m=4
adjusted =
[25.694050712757576,25.02878897387461,28.940819267265272,31.27093608902728,34.25873428367677,35.755412819820876,38.9204121180464,41.35833482742
last = 51.44573357
  by: Seasonally adjusted series (multiplicative)  (seasonal indices of classical decomposition, then x_t / S_{season(t)})
  assumes: n >= 2m; trend: centred moving average of length m (2 x m for even m); index i belongs to the season of observation i
(the list starts at the first observation's season); positive data
```

What the worked example means: This calculation produces the seasonally adjusted series [25.6941, 25.0288, 28.9408, 31.2709, 34.2587, 35.7554, 38.9204, 41.3583, 42.8234, 46.482, 47.902, 51.4457] by dividing each observation in the input series [30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51] by its corresponding seasonal index. The process assumes a multiplicative relationship where seasonal fluctuations scale with the level of the data. By removing these predictable patterns, the result reveals the underlying trend and irregular components, with the most recent adjusted value reported as 51.4457.

Cross-correlation (2)

Lag of the strongest cross-correlation — `cross_correlation_best_lag`

The lag k in -K..K with the largest |r_xy(k)|, r_xy(k) = corr(x_{t+k}, y_t): a positive k means y leads x by k periods.

$$k^* = \arg\max_{|k| \le K} |r_{xy}(k)|$$

symbol		meaning	SI unit
`x`	in	the first series	—
`y`	in	the second series (same length)	—
`K`	in	the largest lag searched	—
`lag`	out	the lag with the largest absolute cross-correlation	—
`ccf`	out	r_xy at that lag	—

Applies when: equal lengths; K <= n - 1; ties: the smallest |k|, then the positive k.

```
$ time-series solve cross_correlation_best_lag "x=[1, 3, 2, 5, 4, 6, 5, 8]" "y=[3, 2, 5, 4, 6, 5, 8, 7]" K=3
lag = 1
ccf = 0.7136566647
  by: Lag of the strongest cross-correlation  (r_xy(k) for every k in -K..K, the largest in absolute value)
  assumes: equal lengths; K <= n - 1; ties: the smallest |k|, then the positive k
```

What the worked example means: This calculation identifies the specific time delay where the correlation between the first series and the second series is most pronounced in absolute magnitude. By evaluating the cross-correlation function across all possible lags within the specified search range, the method determines that the peak association happens at 1 periods. The strength of this relationship is quantified by the cross-correlation coefficient of 0.713657, indicating how closely the movements of one series align with the other when shifted by this amount.

Sample cross-correlation at lag k — `cross_correlation_at_lag`

The sample cross-correlation r_xy(k) = corr(x_{t+k}, y_t) of two equally long series (y leads x by k).

$$r_{xy}(k) = \frac{\frac{1}{n}\sum_{t=1}^{n-k}(x_{t+k} - \bar{x})(y_t - \bar{y})}{s_x s_y}$$

symbol		meaning	SI unit
`x`	in	the first series	—
`y`	in	the second series (same length)	—
`k`	in	the lag k >= 0	—
`ccf`	out	the cross-correlation r_xy(k)	—

Applies when: equal lengths; 0 <= k <= n - 1; denominator n, standard deviations with divisor n.

```
$ time-series solve cross_correlation_at_lag "x=[1, 3, 2, 5, 4, 6]" "y=[2, 1, 4, 3, 6, 5]" k=1
ccf = 0.6428571429
  by: Sample cross-correlation at lag k  (r_xy(k) = (1/n) sum_t (x_{t+k} - xbar)(y_t - ybar) / (s_x s_y))
  assumes: equal lengths; 0 <= k <= n - 1; denominator n, standard deviations with divisor n
```

What the worked example means: This calculation determines the sample cross-correlation between two time series, x and y, at a specific lag k. The result, 0.642857, indicates how strongly values in series x relate to values in series y when y leads x by 1 periods. A value close to one suggests a strong positive linear relationship at that delay, while a value near zero implies little to no linear association.

Exponential smoothing (8)

Best smoothing constant among candidates — `ses_best_alpha`

Among candidate smoothing constants, the one whose simple exponential smoothing (l_0 = x_1) gives the smallest sum of squared one-step errors, with that SSE and its forecast.

$$\alpha^* = \arg\min_{\alpha \in A} \sum_t (x_t - \ell_{t-1}(\alpha))^2$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`alphas`	in	candidate smoothing constants, each in (0, 1]	—
`alpha`	out	the best smoothing constant	—
`sse`	out	its sum of squared one-step errors	—
`forecast`	out	its forecast of the next period	—

Applies when: l_0 = x_1; ties: the first candidate listed.

```
$ time-series solve ses_best_alpha "alphas=[0.1, 0.3, 0.5]" "x=[20, 22, 21, 24, 23, 26]"
alpha = 0.5
sse = 23.8125
forecast = 24.375
  by: Best smoothing constant among candidates  (simple exponential smoothing for each candidate; the smallest SSE)
  assumes: l_0 = x_1; ties: the first candidate listed
```

What the worked example means: Simple exponential smoothing was applied to the series [20, 22, 21, 24, 23, 26] for each candidate smoothing constant in [0.1, 0.3, 0.5]. The method calculated the one-step ahead forecasts starting with an initial level equal to the first observation. The result shows that 0.5 produced the smallest sum of squared errors, specifically 23.8125. Using this optimal constant, the forecast for the next time period is 24.375. This selection minimizes the historical prediction error among the provided options.

Damped-trend Holt method — `holt_damped_trend`

Holt's method with a damped trend (damping parameter phi): the h-step forecast is l_n + (phi + phi^2 + ... + phi^h) b_n. Initial states l_0 = x_1, b_0 = x_2 - x_1 unless stated.

$$\hat{x}_{n+h} = \ell_n + (\phi + \phi^2 + \cdots + \phi^h)\, b_n$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`alpha`	in	level smoothing constant, 0 < alpha <= 1	—
`beta`	in	trend smoothing constant, 0 < beta <= 1 (b_t = beta (l_t - l_{t-1}) + (1 - beta) b_{t-1})	—
`phi`	in	damping parameter, 0 < phi <= 1	—
`l0`	in	initial level l_0 (default x_1) (optional)	—
`b0`	in	initial trend b_0 (default x_2 - x_1) (optional)	—
`h`	in	forecast horizon in periods (default 1) (optional)	—
`forecast`	out	the h-step-ahead forecast	—
`level`	out	the final level l_n	—
`trend`	out	the final trend b_n	—
`sse`	out	sum of squared one-step errors	—

Applies when: 0 < alpha, beta, phi <= 1; l_0 = x_1, b_0 = x_2 - x_1 unless stated.

```
$ time-series solve holt_damped_trend "x=[10, 12, 15, 16, 19, 21]" alpha=0.5 beta=0.3 phi=0.9
forecast = 21.66485143
level = 20.19273161
trend = 1.635688691
sse = 13.01694759
  by: Damped-trend Holt method  (l_t = alpha x_t + (1-alpha)(l_{t-1} + phi b_{t-1}); b_t = beta (l_t - l_{t-1}) + (1-beta) phi
b_{t-1})
  alpha = 0.5
  beta = 0.3
  phi = 0.9
  assumes: 0 < alpha, beta, phi <= 1; l_0 = x_1, b_0 = x_2 - x_1 unless stated
```

What the worked example means: This calculation applies Holt's method with a damping factor to project the series forward. The model updates the level using the smoothing constant alpha and the trend using beta, while phi dampens the trend effect over time. The resulting h-step-ahead forecast is 21.6649, derived from the final estimated level of 20.1927 and trend of 1.63569. This approach assumes that any current trend will gradually weaken rather than persist indefinitely, making it suitable for series where growth or decline is expected to stabilize.

Exponential smoothing forecast update — `exponential_smoothing_update`

The next forecast by simple exponential smoothing from the previous forecast, the actual value and the smoothing constant: F_new = F + alpha (A - F).

$$F_{t+1} = F_t + \alpha\,(A_t - F_t)$$

symbol		meaning	SI unit
`F`	in	the previous forecast	—
`A`	in	the actual value of that period	—
`alpha`	in	smoothing constant, 0 < alpha <= 1	—
`forecast`	out	the new forecast	—
`error`	out	the forecast error A - F	—

Applies when: 0 < alpha <= 1.

```
$ time-series solve exponential_smoothing_update F=100 A=110 alpha=0.2
forecast = 102
error = 10
  by: Exponential smoothing forecast update  (F_new = F + alpha (A - F) = alpha A + (1 - alpha) F)
  F = 100
  A = 110
  alpha = 0.2
  assumes: 0 < alpha <= 1
```

What the worked example means: This calculation determines the next period's forecast using simple exponential smoothing. It starts with the previous forecast of 100 and adjusts it by adding a portion of the forecast error, which is the difference between the actual value 110 and the previous forecast. The adjustment size depends on the smoothing constant 0.2. The resulting new forecast is 102, while the raw forecast error was 10. This method allows the model to react quickly to recent changes in data levels without requiring extensive historical storage.

Holt's linear trend method — `holt_linear_trend`

Holt's linear (double) exponential smoothing: level and trend updated each period, h-step forecast l_n + h b_n. Initial states l_0 = x_1 and b_0 = x_2 - x_1 (before the first observation) unless stated.

$$\ell_t = \alpha x_t + (1-\alpha)(\ell_{t-1} + b_{t-1}),\; b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta) b_{t-1},\; \hat{x}_{n+h} = \ell_n + h\,b_n$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`alpha`	in	level smoothing constant, 0 < alpha <= 1	—
`beta`	in	trend smoothing constant, 0 < beta <= 1 (b_t = beta (l_t - l_{t-1}) + (1 - beta) b_{t-1})	—
`l0`	in	initial level l_0 (default x_1) (optional)	—
`b0`	in	initial trend b_0 (default x_2 - x_1) (optional)	—
`h`	in	forecast horizon in periods (default 1) (optional)	—
`forecast`	out	the h-step-ahead forecast	—
`level`	out	the final level l_n	—
`trend`	out	the final trend b_n	—
`sse`	out	sum of squared one-step errors	—

Applies when: 0 < alpha, beta <= 1; l_0 = x_1, b_0 = x_2 - x_1 unless stated; trend form: beta multiplies (l_t - l_{t-1}) (the error-correction beta* of many texts).

```
$ time-series solve holt_linear_trend "x=[10, 12, 15, 16, 19, 21]" alpha=0.5 beta=0.3
forecast = 22.65328591
level = 20.65533969
trend = 1.997946219
sse = 7.492761855
  by: Holt's linear trend method  (l_t = alpha x_t + (1-alpha)(l_{t-1}+b_{t-1}); b_t = beta (l_t - l_{t-1}) + (1-beta) b_{t-1})
  alpha = 0.5
  beta = 0.3
  assumes: 0 < alpha, beta <= 1; l_0 = x_1, b_0 = x_2 - x_1 unless stated; trend form: beta multiplies (l_t - l_{t-1}) (the error-
correction beta* of many texts)
```

What the worked example means: Holt's linear trend method computes a future value by combining the current level with the estimated slope. Using the input series, smoothing constants, and initial states, the algorithm updates these components recursively. The resulting forecast of 22.6533 reflects the expected trajectory after projecting the final level of 20.6553 forward by the specified horizon using the trend of 1.99795.

Holt-Winters additive method — `holt_winters_additive`

Holt-Winters exponential smoothing with additive trend and additive seasonality: the h-step forecast, final level and trend, and the sum of squared one-step errors. initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of season 1)/m, s_j = x_j - l_0 (additive) or x_j / l_0 (multiplicative) for the first m observations.

$$\hat{x}_{n+h} = \ell_n + hb_n + s_{n+h-m(\lfloor (h-1)/m \rfloor + 1)}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (at least two full seasons)	—
`m`	in	season length	—
`alpha`	in	level smoothing constant	—
`beta`	in	trend smoothing constant	—
`gamma`	in	seasonal smoothing constant	—
`h`	in	forecast horizon in periods (default 1) (optional)	—
`forecast`	out	the h-step-ahead forecast	—
`level`	out	final level l_n	—
`trend`	out	final trend b_n	—
`sse`	out	sum of squared one-step errors	—

Applies when: n >= 2m; 0 < alpha, beta, gamma <= 1; initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of season 1)/m, s_j = x_j - l_0 (additive) or x_j / l_0 (multiplicative) for the first m observations.

```
$ time-series solve holt_winters_additive "x=[30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51]" m=4 alpha=0.5 beta=0.3 gamma=0.2
forecast = 56.15883046
level = 49.59834899
trend = 2.048382087
sse = 146.5534279
  by: Holt-Winters additive method  (l_t = alpha (x_t - s_{t-m}) + (1-alpha)(l_{t-1}+b_{t-1}); s_t = gamma (x_t - l_{t-1} - b_{t-
1}) + (1-gamma) s_{t-m}; b_t = beta (l_t - l_{t-1}) + (1-beta) b_{t-1})
  alpha = 0.5
  beta = 0.3
  gamma = 0.2
  assumes: n >= 2m; 0 < alpha, beta, gamma <= 1; initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of
season 1)/m, s_j = x_j - l_0 (additive) or x_j / l_0 (multiplicative) for the first m observations
```

What the worked example means: Holt-Winters additive smoothing computes the forecast using the last estimated level, trend, and seasonal component. The result 56.1588 combines these elements for the next period. The final level 49.5983 represents the deseasonalized baseline, while the trend 2.04838 indicates the average change per period. The sum of squared errors 146.553 measures how well the model fit past data.

Holt-Winters multiplicative method — `holt_winters_multiplicative`

Holt-Winters exponential smoothing with additive trend and multiplicative seasonality: the h-step forecast, final level and trend, and the sum of squared one-step errors. initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of season 1)/m, s_j = x_j - l_0 (additive) or x_j / l_0 (multiplicative) for the first m observations.

$$\hat{x}_{n+h} = (\ell_n + hb_n)\, s_{n+h-m(\lfloor (h-1)/m \rfloor + 1)}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (at least two full seasons)	—
`m`	in	season length	—
`alpha`	in	level smoothing constant	—
`beta`	in	trend smoothing constant	—
`gamma`	in	seasonal smoothing constant	—
`h`	in	forecast horizon in periods (default 1) (optional)	—
`forecast`	out	the h-step-ahead forecast	—
`level`	out	final level l_n	—
`trend`	out	final trend b_n	—
`sse`	out	sum of squared one-step errors	—

Applies when: n >= 2m; 0 < alpha, beta, gamma <= 1; initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of season 1)/m, s_j = x_j - l_0 (additive) or x_j / l_0 (multiplicative) for the first m observations; positive data.

```
$ time-series solve holt_winters_multiplicative "x=[30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51]" m=4 alpha=0.5 beta=0.3
gamma=0.2
forecast = 58.65652262
level = 49.64619786
trend = 1.850979687
sse = 219.2701175
  by: Holt-Winters multiplicative method  (l_t = alpha x_t/s_{t-m} + (1-alpha)(l_{t-1}+b_{t-1}); s_t = gamma x_t/(l_{t-1}+b_{t-1})
+ (1-gamma) s_{t-m}; b_t = beta (l_t - l_{t-1}) + (1-beta) b_{t-1})
  alpha = 0.5
  beta = 0.3
  gamma = 0.2
  assumes: n >= 2m; 0 < alpha, beta, gamma <= 1; initialization: l_0 = mean of the first season, b_0 = (mean of season 2 - mean of
season 1)/m, s_j = x_j - l_0 (additive) or x_j / l_0 (multiplicative) for the first m observations; positive data
```

What the worked example means: This calculation applies Holt-Winters exponential smoothing with multiplicative seasonality to determine future values. Using a series length of at least two full seasons, the method updates the level, trend, and seasonal components recursively. The resulting h-step-ahead forecast of 58.6565 reflects the current level of 49.6462 adjusted by the trend of 1.85098 and the appropriate seasonal factor. The sum of squared one-step errors is 219.27, indicating how well the model fits historical data.

One update of Holt's method — `holt_update`

Update Holt's level and trend with one new observation and forecast h periods ahead.

$$\ell_t = \alpha y_t + (1-\alpha)(\ell_{t-1} + b_{t-1}),\; b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta) b_{t-1}$$

symbol		meaning	SI unit
`l`	in	previous level	—
`b`	in	previous trend	—
`y`	in	the new observation	—
`alpha`	in	level smoothing constant	—
`beta`	in	trend smoothing constant	—
`h`	in	forecast horizon (default 1) (optional)	—
`level`	out	new level	—
`trend`	out	new trend	—
`forecast`	out	level + h trend	—

Applies when: 0 < alpha, beta <= 1; b_t = beta (l_t - l_{t-1}) + (1 - beta) b_{t-1}.

```
$ time-series solve holt_update l=50 b=2 y=55 alpha=0.3 beta=0.1
level = 52.9
trend = 2.09
forecast = 54.99
  by: One update of Holt's method  (l = alpha y + (1 - alpha)(l + b); b = beta (l_new - l) + (1 - beta) b)
  l = 50
  b = 2
  y = 55
  alpha = 0.3
  beta = 0.1
  assumes: 0 < alpha, beta <= 1; b_t = beta (l_t - l_{t-1}) + (1 - beta) b_{t-1}
```

What the worked example means: Holt's method updates the series by calculating a new level and a new trend using the previous estimates and the latest observation. The new level, shown as 52.9, blends the current observation with the prior forecast for this period. The new trend, shown as 2.09, adjusts the previous slope based on the change in level. These updated values produce a forecast of 54.99 for the next time step.

Simple exponential smoothing — `simple_exponential_smoothing`

Simple exponential smoothing with smoothing constant alpha: the forecast of the next period and the sum of squared one-step errors. The initial level l_0 is the first observation unless stated.

$$\ell_t = \alpha x_t + (1-\alpha)\,\ell_{t-1}, \qquad \hat{x}_{n+1} = \ell_n$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`alpha`	in	the smoothing constant, 0 < alpha <= 1	—
`l0`	in	the initial level l_0 before the first observation (default: x_1) (optional)	—
`forecast`	out	the forecast of the next period (the final level l_n)	—
`sse`	out	sum of squared one-step errors, e_t = x_t - l_{t-1}	—

Applies when: 0 < alpha <= 1; l_0 = x_1 unless an initial level is stated.

```
$ time-series solve simple_exponential_smoothing "x=[20, 22, 21, 24, 23]" alpha=0.3
forecast = 22.0928
sse = 16.598016
  by: Simple exponential smoothing  (l_t = l_{t-1} + alpha (x_t - l_{t-1}); forecast = l_n)
  alpha = 0.3
  assumes: 0 < alpha <= 1; l_0 = x_1 unless an initial level is stated
```

What the worked example means: Simple exponential smoothing computes a forecast by updating a single level estimate using recent observations weighted by the smoothing constant alpha. Starting from an initial level, each new data point adjusts the previous level to produce the final forecast value of 22.0928. The method also calculates the sum of squared one-step prediction errors, which equals 16.598, providing a measure of how well the model tracked the historical series.

Forecast accuracy (7)

Mean absolute error — `mean_absolute_error`

The mean absolute error of forecasts against actual values.

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - f_t|$$

symbol		meaning	SI unit
`y`	in	the actual values	—
`f`	in	the forecasts (same length)	—
`mae`	out	Mean absolute error	—
`n`	out	number of forecasts	—

Applies when: equal lengths; errors e_t = actual - forecast.

```
$ time-series solve mean_absolute_error "y=[10, 12, 14, 13]" "f=[11, 12, 13, 15]"
mae = 1
n = 4
  by: Mean absolute error  (mean of |actual - forecast|)
  assumes: equal lengths; errors e_t = actual - forecast
```

What the worked example means: The mean absolute error measures the average magnitude of errors between actual values and forecasts, ignoring their direction. It is computed by taking the absolute difference for each time step, summing these differences, and dividing by the total number of observations, which here is 4. The resulting value, 1, represents the typical size of a forecast error in the same units as the original data.

Mean absolute percentage error — `mean_absolute_percentage_error`

The mean absolute percentage error (in percent) of forecasts against actual values.

$$\mathrm{MAPE} = \frac{100}{n}\sum_{t=1}^{n}\left|\frac{y_t - f_t}{y_t}\right|$$

symbol		meaning	SI unit
`y`	in	the actual values	—
`f`	in	the forecasts (same length)	—
`mape`	out	Mean absolute percentage error	—
`n`	out	number of forecasts	—

Applies when: equal lengths; errors e_t = actual - forecast; no actual value is 0; the result is in percent.

```
$ time-series solve mean_absolute_percentage_error "y=[10, 12, 14, 13]" "f=[11, 12, 13, 15]"
mape = 8.131868132
n = 4
  by: Mean absolute percentage error  (100 x mean of |actual - forecast| / |actual|)
  assumes: equal lengths; errors e_t = actual - forecast; no actual value is 0; the result is in percent
```

What the worked example means: This calculation determines the mean absolute percentage error by averaging the absolute differences between actual values and forecasts, divided by the actual values, then multiplying by one hundred. The result, expressed as 8.13187, represents the average magnitude of forecast errors in relative terms across the 4 data points provided in the input series.

Mean absolute scaled error — `mean_absolute_scaled_error`

MASE: the forecasts' mean absolute error divided by the in-sample mean absolute error of the (seasonal) naive forecast on the training data.

$$\mathrm{MASE} = \frac{\frac{1}{h}\sum|y_t - f_t|}{\frac{1}{T-m}\sum_{t=m+1}^{T}|y_t - y_{t-m}|}$$

symbol		meaning	SI unit
`y_train`	in	the training (in-sample) data	—
`y`	in	the actual values	—
`f`	in	the forecasts (same length as y)	—
`sp`	in	seasonal period of the naive benchmark (default 1) (optional)	—
`mase`	out	the mean absolute scaled error	—
`mae`	out	the forecasts' MAE	—
`scale`	out	in-sample MAE of the lag-sp naive forecast	—

Applies when: the training data are longer than sp and not constant at lag sp.

```
$ time-series solve mean_absolute_scaled_error "y_train=[10, 12, 11, 14, 13]" "y=[15, 14, 16]" "f=[14, 15, 15]"
mase = 0.5714285714
mae = 1
scale = 1.75
  by: Mean absolute scaled error  (MAE(actual, forecast) / mean |y_train_t - y_train_{t-sp}|)
  assumes: the training data are longer than sp and not constant at lag sp
```

What the worked example means: This metric compares the accuracy of your forecasts against a simple naive benchmark. It takes the mean absolute error of the predictions, which is 1, and divides it by the mean absolute error of the seasonal naive forecast on the training data, which is 1.75. The resulting value, 0.571429, indicates relative performance; a value less than one suggests the model outperforms the naive baseline, while a value greater than one indicates it performs worse.

Mean forecast error (bias) — `mean_forecast_error`

The mean forecast error (bias): positive when the forecasts are too low on average.

$$\mathrm{ME} = \frac{1}{n}\sum_{t=1}^{n}(y_t - f_t)$$

symbol		meaning	SI unit
`y`	in	the actual values	—
`f`	in	the forecasts (same length)	—
`mean_error`	out	Mean forecast error (bias)	—
`n`	out	number of forecasts	—

Applies when: equal lengths; errors e_t = actual - forecast.

```
$ time-series solve mean_forecast_error "y=[10, 12, 14, 13]" "f=[11, 12, 13, 15]"
mean_error = -0.5
n = 4
  by: Mean forecast error (bias)  (mean of actual - forecast (computed by the reference as mean(actual) - mean(forecast)))
  assumes: equal lengths; errors e_t = actual - forecast
```

What the worked example means: This calculation determines the bias of the forecasting model by computing the average of the differences between the actual observed values and the predicted forecasts. The result, -0.5, represents the mean forecast error over 4 periods. A positive value suggests that the model tends to under-forecast, meaning the actual values were higher than predicted on average, while a negative value indicates over-forecasting. This metric relies on the assumption that the error term is defined as the actual value minus the forecast value.

Mean squared error — `mean_squared_error`

The mean squared error of forecasts against actual values.

$$\mathrm{MSE} = \frac{1}{n}\sum_{t=1}^{n}(y_t - f_t)^2$$

symbol		meaning	SI unit
`y`	in	the actual values	—
`f`	in	the forecasts (same length)	—
`mse`	out	Mean squared error	—
`n`	out	number of forecasts	—

Applies when: equal lengths; errors e_t = actual - forecast.

```
$ time-series solve mean_squared_error "y=[10, 12, 14, 13]" "f=[11, 12, 13, 15]"
mse = 1.5
n = 4
  by: Mean squared error  (mean of (actual - forecast)^2)
  assumes: equal lengths; errors e_t = actual - forecast
```

What the worked example means: This calculation determines the average of the squared differences between the actual values and the forecasts. It uses [10, 12, 14, 13] for the observed data and [11, 12, 13, 15] for the predicted values, resulting in a mean squared error of 1.5. This metric penalizes larger errors more severely than smaller ones, providing a measure of how far off the predictions are on average across the 4 time steps analyzed.

Root mean squared error — `root_mean_squared_error`

The root mean squared error of forecasts against actual values.

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - f_t)^2}$$

symbol		meaning	SI unit
`y`	in	the actual values	—
`f`	in	the forecasts (same length)	—
`rmse`	out	Root mean squared error	—
`n`	out	number of forecasts	—

Applies when: equal lengths; errors e_t = actual - forecast.

```
$ time-series solve root_mean_squared_error "y=[10, 12, 14, 13]" "f=[11, 12, 13, 15]"
rmse = 1.224744871
n = 4
  by: Root mean squared error  (square root of the mean squared error)
  assumes: equal lengths; errors e_t = actual - forecast
```

What the worked example means: This calculation determines the typical magnitude of forecast errors by taking the square root of the average squared differences between actual values and forecasts. The result, 1.22474, represents the standard deviation-like measure of these deviations in the original units of the data. It was derived using 4 observations where each error was computed as the actual value minus the forecast value. A lower 1.22474 indicates that forecasts are generally closer to the actual outcomes, providing a single summary statistic for overall accuracy.

Symmetric mean absolute percentage error — `symmetric_mape`

The symmetric MAPE in percent: 100/n sum 2|y - f| / (|y| + |f|).

$$\mathrm{sMAPE} = \frac{100}{n}\sum_{t=1}^{n}\frac{2|y_t - f_t|}{|y_t| + |f_t|}$$

symbol		meaning	SI unit
`y`	in	the actual values	—
`f`	in	the forecasts (same length)	—
`smape`	out	Symmetric mean absolute percentage error	—
`n`	out	number of forecasts	—

Applies when: equal lengths; errors e_t = actual - forecast; no pair with actual and forecast both 0; the result is in percent (range 0 to 200).

```
$ time-series solve symmetric_mape "y=[10, 12, 14, 13]" "f=[11, 12, 13, 15]"
smape = 7.804232804
n = 4
  by: Symmetric mean absolute percentage error  (100 x mean of 2|actual - forecast| / (|actual| + |forecast|))
  assumes: equal lengths; errors e_t = actual - forecast; no pair with actual and forecast both 0; the result is in percent (range
0 to 200)
```

What the worked example means: This calculation determines the symmetric mean absolute percentage error, which serves as a measure of forecast accuracy. It is computed by taking the average of the absolute difference between each actual value and its corresponding forecast, scaled by the sum of their absolute magnitudes, and then multiplied by one hundred to express the result as a percentage. The final score of 7.80423 indicates how well the forecasts matched the observed data across 4 time points, with lower values representing better predictive performance.

Interpolation of missing values (2)

Exponential (log-linear) interpolation — `exponential_interpolation`

Fill a missing value at time t0 assuming constant growth between the neighbouring observations (linear interpolation of the logarithms).

$$\hat{y}(t_0) = y_i\left(\frac{y_{i+1}}{y_i}\right)^{\frac{t_0 - t_i}{t_{i+1} - t_i}}$$

symbol		meaning	SI unit
`t`	in	the times (strictly increasing)	—
`y`	in	the values at those times	—
`t0`	in	the time of the missing value	—
`value`	out	the interpolated value at t0	—

Applies when: positive values; times strictly increasing; t0 within the observed times.

```
$ time-series solve exponential_interpolation "t=[1, 2, 3, 4]" "y=[100, 120, 150, 170]" t0=2.5
value = 134.1640786
  by: Exponential (log-linear) interpolation  (y_i (y_{i+1}/y_i)^((t0 - t_i)/(t_{i+1} - t_i)))
  t0 = 2.5
  assumes: positive values; times strictly increasing; t0 within the observed times
```

What the worked example means: Exponential interpolation estimates the missing value at time 2.5 by assuming a constant growth rate between the nearest known points. Using the observed values and times, the calculation yields 134.164. This result represents the likely magnitude of the series at that specific moment, reflecting multiplicative rather than additive changes.

Linear interpolation of a missing value — `linear_interpolation`

Fill a missing value at time t0 by the straight line between the neighbouring observations.

$$\hat{y}(t_0) = y_i + \frac{t_0 - t_i}{t_{i+1} - t_i}(y_{i+1} - y_i)$$

symbol		meaning	SI unit
`t`	in	the times (strictly increasing)	—
`y`	in	the values at those times	—
`t0`	in	the time of the missing value	—
`value`	out	the interpolated value at t0	—

Applies when: times strictly increasing; t0 within the observed times (no extrapolation).

```
$ time-series solve linear_interpolation "t=[1, 2, 3, 4]" "y=[10, 14, 13, 18]" t0=2.5
value = 13.5
  by: Linear interpolation of a missing value  (y_i + (t0 - t_i)(y_{i+1} - y_i)/(t_{i+1} - t_i) with t_i <= t0 <= t_{i+1})
  t0 = 2.5
  assumes: times strictly increasing; t0 within the observed times (no extrapolation)
```

What the worked example means: Linear interpolation estimates the missing data point by calculating a weighted average of the two nearest known observations. The result, 13.5, represents the position on the straight line connecting these neighbors at the specific time 2.5. This method assumes the trend between the points is linear and provides a continuous estimate within the observed range.

Kalman filtering (7)

Kalman filter for a constant-velocity model — `kalman_filter_constant_velocity`

Track position and velocity from noisy position measurements with the constant-velocity Kalman filter (discrete white-noise acceleration with variance q, time step dt): final state and covariance.

$$F = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix},\ Q = q \begin{pmatrix} \Delta t^4/4 & \Delta t^3/2 \\ \Delta t^3/2 & \Delta t^2 \end{pmatrix}$$

symbol		meaning	SI unit
`x`	in	the measured positions, oldest first	—
`dt`	in	time step	—
`q`	in	acceleration (process) noise variance	—
`r`	in	position measurement noise variance	—
`p0`	in	initial position estimate	—
`v0`	in	initial velocity estimate	—
`var0`	in	initial variance of position and of velocity (uncorrelated)	—
`position`	out	final position estimate	—
`velocity`	out	final velocity estimate	—
`p11`	out	final position variance	—
`p12`	out	final position-velocity covariance	—
`p22`	out	final velocity variance	—

Applies when: F = [[1, dt], [0, 1]], H = [1, 0]; Q = q [[dt^4/4, dt^3/2], [dt^3/2, dt^2]]; predict then update at every measurement.

```
$ time-series solve kalman_filter_constant_velocity "x=[1.1, 1.9, 3.2, 4, 4.8]" dt=1 q=0.1 r=1 p0=0 v0=1 var0=10
position = 4.896949573
velocity = 0.9385623391
p11 = 0.6153900119
p12 = 0.2443095291
p22 = 0.224491015
  by: Kalman filter for a constant-velocity model  (2-state Kalman filter recursion)
  dt = 1
  q = 0.1
  r = 1
  p0 = 0
  v0 = 1
  var0 = 10
  assumes: F = [[1, dt], [0, 1]], H = [1, 0]; Q = q [[dt^4/4, dt^3/2], [dt^3/2, dt^2]]; predict then update at every measurement
```

What the worked example means: This algorithm tracks the state of an object moving with constant velocity by processing noisy measurements. Starting from initial estimates of position 0 and velocity 1, it iteratively predicts the next state and updates it with each measurement. The process accounts for random acceleration noise with variance 0.1 and measurement noise with variance 1 over time steps of 1. The result is a refined estimate of the current position at 4.89695 and velocity at 0.938562, along with their associated uncertainties.

Kalman filter for a random-walk (local level) model — `kalman_filter_local_level`

Filter a sequence of measurements with the scalar Kalman filter for x_t = x_{t-1} + w_t, z_t = x_t + v_t (predict, then update, at every step): the final estimate, variance, gain, all filtered estimates and the Gaussian log-likelihood of the one-step prediction errors.

$$P_t^- = P_{t-1} + q,\ K_t = \frac{P_t^-}{P_t^- + r},\ \hat{x}_t = \hat{x}_{t-1} + K_t(z_t - \hat{x}_{t-1})$$

symbol		meaning	SI unit
`x`	in	the measurements, oldest first	—
`q`	in	process noise variance	—
`r`	in	measurement noise variance	—
`m0`	in	initial state estimate	—
`p0`	in	initial variance	—
`estimate`	out	final filtered estimate	—
`variance`	out	final variance	—
`gain`	out	final gain	—
`estimates`	out	filtered estimates for every measurement	—
`log_likelihood`	out	sum of log N(v_t; 0, S_t)	—

Applies when: q >= 0, r > 0, p0 >= 0; the initial estimate is the state before the first measurement.

```
$ time-series solve kalman_filter_local_level "x=[10.4, 11.1, 10.7, 11.9, 12.3]" q=0.5 r=2 m0=10 p0=5
estimate = 11.62410315
variance = 0.7906396594
gain = 0.3953198297
estimates = [10.293333333333333,10.693277310924369,10.696149217809866,11.182224215246636,11.624103148133084]
log_likelihood = -8.617118062
  by: Kalman filter for a random-walk (local level) model  (P- = P + q; S = P- + r; K = P-/S; x = x + K(z - x); P = (1 - K) P-)
  q = 0.5
  r = 2
  m0 = 10
  p0 = 5
  assumes: q >= 0, r > 0, p0 >= 0; the initial estimate is the state before the first measurement
```

What the worked example means: This algorithm computes the state of a random walk by recursively updating an estimate and its uncertainty. Starting from initial guess 10 and variance 5, it processes measurements to produce a final estimate of 11.6241. The result reflects a balance between process noise 0.5 and measurement noise 2, yielding a final gain of 0.39532 that determines how much weight new data carries relative to prior belief.

Kalman measurement update (scalar) — `kalman_update`

Combine a prior estimate (mean, variance) with a measurement (value, noise variance): the posterior mean and variance and the Kalman gain.

$$K = \frac{P}{P + R},\ \hat{x}^+ = \hat{x} + K(z - \hat{x}),\ P^+ = (1 - K)P$$

symbol		meaning	SI unit
`m`	in	prior mean	—
`p`	in	prior variance (> 0)	—
`z`	in	the measurement	—
`r`	in	measurement noise variance (> 0)	—

symbol		meaning	SI unit
`mean`	out	posterior mean	—
`variance`	out	posterior variance	—
`gain`	out	Kalman gain p/(p+r)	—

Applies when: p > 0, r > 0; the measurement observes the state directly (H = 1).

```
$ time-series solve kalman_update m=10 p=4 z=12 r=1
mean = 11.6
variance = 0.8
gain = 0.8
  by: Kalman measurement update (scalar)  (K = p/(p+r); m' = m + K(z - m); p' = (1 - K) p)
  m = 10
  p = 4
  z = 12
  r = 1
  assumes: p > 0, r > 0; the measurement observes the state directly (H = 1)
```

What the worked example means: This calculation updates the state estimate by blending the prior mean 10 with the new measurement 12. The weight given to the measurement, known as the Kalman gain, is computed as 0.8 based on the relative uncertainties of the prior variance 4 and the measurement noise variance 1. The resulting posterior mean 11.6 represents the refined belief about the state, while the posterior variance 0.8 indicates the reduced uncertainty after incorporating this new data point.

One Kalman filter step (random-walk model) — `kalman_step`

One predict-and-update step of the scalar Kalman filter for a random-walk state x_t = x_{t-1} + w_t observed as z_t = x_t + v_t.

$$P^- = P + Q,\ K = \frac{P^-}{P^- + R},\ \hat{x} = \hat{x}^- + K(z - \hat{x}^-),\ P = (1 - K)P^-$$

symbol		meaning	SI unit
`m`	in	previous estimate	—
`p`	in	previous estimate variance	—
`z`	in	the new measurement	—
`q`	in	process noise variance	—
`r`	in	measurement noise variance	—
`mean`	out	updated estimate	—
`variance`	out	updated variance	—
`gain`	out	Kalman gain	—
`prior_variance`	out	predicted variance p + q	—

Applies when: p, q >= 0, r > 0; F = H = 1.

```
$ time-series solve kalman_step m=10 p=4 z=12 q=1 r=2
mean = 11.42857143
variance = 1.428571429
gain = 0.7142857143
prior_variance = 5
  by: One Kalman filter step (random-walk model)  (predict P- = P + Q; K = P-/(P- + R); x = x + K(z - x); P = (1 - K) P-)
  m = 10
  p = 4
  z = 12
  q = 1
  r = 2
  assumes: p, q >= 0, r > 0; F = H = 1
```

What the worked example means: This calculation performs one step of the scalar Kalman filter for a random-walk model. It starts with the previous estimate 10 and its variance 4, then predicts forward by adding process noise variance 1 to get a prior variance of 5. Upon receiving measurement 12 with noise variance 2, it computes the Kalman gain 0.714286 to weigh the new information against the prediction. The final result is an updated state estimate of 11.4286 and a reduced uncertainty variance of 1.42857.

Rauch-Tung-Striebel smoother (random-walk model) — `rts_smoother_local_level`

Smoothed state estimates of the random-walk (local level) model: the Kalman filter forward, then the Rauch-Tung-Striebel recursion backward.

$$\hat{x}_{t|n} = \hat{x}_t + C_t(\hat{x}_{t+1|n} - \hat{x}_t),\ C_t = \frac{P_t}{P_t + q}$$

symbol		meaning	SI unit
`x`	in	the measurements, oldest first	—
`q`	in	process noise variance	—
`r`	in	measurement noise variance	—

symbol		meaning	SI unit
`m0`	in	initial state estimate	—
`p0`	in	initial variance	—
`smoothed`	out	smoothed estimates for every time	—
`first`	out	smoothed estimate at time 1	—
`first_variance`	out	its variance	—

Applies when: q > 0, r > 0, p0 >= 0.

```
$ time-series solve rts_smoother_local_level "x=[10.4, 11.1, 10.7, 11.9, 12.3]" q=0.5 r=2 m0=10 p0=5
smoothed = [10.83039127959001,11.013479215813877,11.174936955991214,11.455128935166355,11.624103148133084]
first = 10.83039128
first_variance = 0.6949754603
  by: Rauch-Tung-Striebel smoother (random-walk model)  (C_t = P_t/(P_t + q); xs_t = x_t + C_t (xs_{t+1} - x_t); Ps_t = P_t +
C_t^2 (Ps_{t+1} - P_t - q))
  q = 0.5
  r = 2
  m0 = 10
  p0 = 5
  assumes: q > 0, r > 0, p0 >= 0
```

What the worked example means: This calculation produces smoothed state estimates for a random walk model by combining forward filtering with backward smoothing. The result shows that the refined estimate for the initial state is 10.8304, accompanied by an uncertainty measure of 0.694975. This value incorporates information from all subsequent observations to provide a more accurate trajectory than the filter alone.

Steady-state Kalman gain (random-walk model) — `kalman_steady_state_gain`

The limiting Kalman gain and variances of the random-walk (local level) model with process variance q and measurement variance r (the solution of the scalar Riccati equation).

$$P^{-} = \frac{q + \sqrt{q^2 + 4qr}}{2}, \quad K = \frac{P^{-}}{P^{-} + r}$$

symbol		meaning	SI unit
`q`	in	process noise variance (> 0)	—
`r`	in	measurement noise variance (> 0)	—
`gain`	out	steady-state gain	—
`prior_variance`	out	steady-state predicted variance P-	—
`posterior_variance`	out	steady-state filtered variance	—

Applies when: q > 0, r > 0.

```
$ time-series solve kalman_steady_state_gain q=1 r=4
gain = 0.3903882032
prior_variance = 2.561552813
posterior_variance = 1.561552813
  by: Steady-state Kalman gain (random-walk model)  (P- = (q + sqrt(q^2 + 4 q r))/2, K = P-/(P- + r))
  q = 1
  r = 4
  assumes: q > 0, r > 0
```

What the worked example means: This calculation determines the long-term behavior of a random-walk model by finding the constant values the filter approaches. Using a process noise variance of 1 and a measurement noise variance of 4, the algorithm computes the steady-state prior variance as 2.56155. This leads to a final gain of 0.390388, which dictates how much weight future measurements receive compared to predictions, resulting in a posterior variance of 1.56155.

Steady-state Kalman gains of a constant-velocity model — `kalman_steady_state_constant_velocity`

The limiting position and velocity gains (and predicted position variance) of the constant-velocity Kalman filter with discrete white-noise acceleration q, time step dt and measurement variance r.

$$P = F\big(P - PH^{\top}(HPH^{\top} + R)^{-1}HP\big)F^{\top} + Q$$

symbol		meaning	SI unit
`dt`	in	time step	—
`q`	in	acceleration noise variance (> 0)	—
`r`	in	measurement noise variance (> 0)	—
`k_position`	out	steady-state position gain	—
`k_velocity`	out	steady-state velocity gain	—
`prior_p11`	out	steady-state predicted position variance	—

Applies when: dt, q, r > 0; Q = q [[dt^4/4, dt^3/2], [dt^3/2, dt^2]].

```
$ time-series solve kalman_steady_state_constant_velocity dt=1 q=0.1 r=1
k_position = 0.5462107896
k_velocity = 0.2130232875
prior_p11 = 1.203666322
  by: Steady-state Kalman gains of a constant-velocity model  (iterate the Riccati recursion to its fixed point)
  dt = 1
  q = 0.1
  r = 1
  assumes: dt, q, r > 0; Q = q [[dt^4/4, dt^3/2], [dt^3/2, dt^2]]
```

What the worked example means: This calculation determines the long-term stable weights for a constant-velocity Kalman filter. Given a time step of 1, acceleration noise variance of 0.1, and measurement noise variance of 1, the algorithm iterates the Riccati equation to find the fixed point. The resulting position gain of 0.546211 shows the fraction of measurement error applied to the position estimate, while the velocity gain of 0.213023 applies that correction to velocity. The predicted position variance settles at 1.20367, reflecting the uncertainty before incorporating the next observation.

Moving averages (4)

Centred moving average — `centred_moving_average`

The centred moving average of length k: a symmetric k-term mean for odd k, the 2 x k moving average (half weights at both ends) for even k, as used in classical decomposition.

$$\mathrm{CMA}_t = \frac{1}{k}\left(\tfrac{1}{2}x_{t-h} + \sum_{i=-h+1}^{h-1} x_{t+i} + \tfrac{1}{2}x_{t+h}\right)$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`k`	in	the length of the moving average	—
`cma`	out	the centred moving averages, from time first_index on	—
`first_index`	out	the time t (1-based) of the first centred value (k//2 + 1)	—
`first`	out	the first centred moving average	—

Applies when: k >= 2; odd k: k <= n; even k: k + 1 <= n.

```
$ time-series solve centred_moving_average "x=[12, 15, 14, 18, 20, 19, 23, 25]" k=4
cma = [15.75,17.25,18.875,20.875]
first_index = 3
first = 15.75
  by: Centred moving average  (odd k: mean of x_{t-h}..x_{t+h}; even k: (x_{t-h}/2 + x_{t-h+1} + ... + x_{t+h-1} + x_{t+h}/2)/k, h
= k//2)
  assumes: k >= 2; odd k: k <= n; even k: k + 1 <= n
```

What the worked example means: This calculation determines the trend-cycle component by applying a symmetric moving average of length 4 to the series. For odd lengths, it takes the simple mean of the surrounding points; for even lengths, it applies half-weights to the endpoints to ensure integer alignment with the original data indices. The resulting value 15.75 represents the smoothed level at time 3, effectively removing short-term fluctuations while preserving the central timing of the window.

Exponential moving average — `exponential_moving_average`

The exponential moving average with span k (alpha = 2/(k+1)), started at the first value.

$$\mathrm{EMA}_t = \alpha x_t + (1-\alpha)\,\mathrm{EMA}_{t-1}, \quad \alpha = \frac{2}{k+1}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`k`	in	the span (number of periods)	—
`ema`	out	the exponential moving average at every time (EMA_1 = x_1)	—
`last`	out	the latest exponential moving average	—
`alpha`	out	the weight 2/(k+1)	—

Applies when: k >= 1; EMA_1 = x_1 (no averaging of a start-up window).

```
$ time-series solve exponential_moving_average "x=[10, 12, 14, 13, 15]" k=3
ema = [10.0,11.0,12.5,12.75,13.875]
last = 13.875
alpha = 0.5
  by: Exponential moving average  (EMA_t = alpha x_t + (1 - alpha) EMA_{t-1}, alpha = 2/(k+1))
  assumes: k >= 1; EMA_1 = x_1 (no averaging of a start-up window)
```

What the worked example means: This calculation produces an exponential moving average series starting from the first value. The latest estimate is 13.875, derived using a smoothing factor of 0.5. This method gives more weight to recent observations while retaining some influence from earlier data points, resulting in a smoother trend line than raw data.

Linearly weighted moving average — `linearly_weighted_moving_average`

The k-period linearly weighted moving average: weights 1, 2, ..., k from the oldest to the newest value of each window.

$$\mathrm{WMA}_t = \frac{\sum_{i=0}^{k-1} (k-i)\, x_{t-i}}{k(k+1)/2}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`k`	in	the window length	—
`lwma`	out	the weighted moving averages, one per complete window	—
`last`	out	the latest weighted moving average (forecast of the next period)	—

Applies when: 1 <= k <= n; the newest value of a window has weight k, the oldest weight 1.

```
$ time-series solve linearly_weighted_moving_average "x=[10, 12, 14, 13, 15]" k=3
lwma = [12.666666666666666,13.166666666666666,14.166666666666666]
last = 14.16666667
  by: Linearly weighted moving average  (weighted mean of each window with weights 1..k divided by k(k+1)/2)
  assumes: 1 <= k <= n; the newest value of a window has weight k, the oldest weight 1
```

What the worked example means: This calculation determines the linearly weighted moving average, denoted as [12.6667, 13.1667, 14.1667], using a window length of 3 applied to the series [10, 12, 14, 13, 15]. The method assigns increasing weights from oldest to newest data points, ensuring recent observations influence the result more strongly. The final computed value, 14.1667, represents the most current smoothed estimate and serves as the forecast for the next period.

Simple moving average — `simple_moving_average`

The k-period simple (trailing) moving average of a series; its last value is the moving-average forecast of the next period.

$$\mathrm{SMA}_t = \frac{1}{k} \sum_{i=0}^{k-1} x_{t-i}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`k`	in	the window length (number of periods)	—
`sma`	out	the moving averages, one per complete window (the first covers x_1..x_k)	—
`last`	out	the latest moving average (forecast of the next period)	—
`count`	out	number of moving averages (n - k + 1)	—

Applies when: 1 <= k <= n.

```
$ time-series solve simple_moving_average "x=[10, 12, 14, 13, 15]" k=3
sma = [12.0,13.0,14.0]
last = 14
count = 3
  by: Simple moving average  (mean of each window of k consecutive values)
  assumes: 1 <= k <= n
```

What the worked example means: This calculation determines the arithmetic mean of the most recent 3 observations from the series. The resulting sequence contains 3 values, where each entry smooths short-term fluctuations to reveal underlying trends. The final computed average, 14, serves as the baseline prediction for the subsequent time step, assuming that recent history is the best indicator of near-future behavior.

Stationarity tests (3)

Augmented Dickey-Fuller statistic — `augmented_dickey_fuller_test`

The augmented Dickey-Fuller tau statistic with p lagged differences: the t-ratio of gamma in Delta x_t = gamma x_{t-1} + sum_j delta_j Delta x_{t-j} (+ constant, + trend) + e_t. The statistic only.

$$\Delta x_t = \alpha + \gamma x_{t-1} + \sum_{j=1}^{p} \delta_j \Delta x_{t-j} + e_t$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—

symbol		meaning	SI unit
`p`	in	number of lagged differences	—
`regression`	in	deterministic terms: c = constant (default), ct = constant and trend, n = none (optional)	—
`tau`	out	the ADF statistic	—
`gamma`	out	the coefficient of x_{t-1}	—
`se`	out	its standard error	—
`nobs`	out	observations in the regression (n - 1 - p)	—

Applies when: p <= n/2 - (deterministic terms) - 1; default deterministic terms: a constant; the lag length is given, not selected.

```
$ time-series solve augmented_dickey_fuller_test "x=[10.2, 10.8, 10.5, 11.3, 11, 11.9, 11.6, 12.4, 12, 12.9, 12.5, 13.4]" p=2
regression=c
tau = -1.186531751
gamma = -0.03905259823
se = 0.0329132349
nobs = 9
  by: Augmented Dickey-Fuller statistic  (least squares on t = p+2..n of Delta x_t on x_{t-1}, Delta x_{t-1..t-p} (and constant /
trend))
  regression = c
  assumes: p <= n/2 - (deterministic terms) - 1; default deterministic terms: a constant; the lag length is given, not selected
```

What the worked example means: This calculation determines the Augmented Dickey-Fuller tau statistic for the time series [10.2, 10.8, 10.5, 11.3, 11, 11.9, 11.6, 12.4, 12, 12.9, 12.5, 13.4]. It performs a least squares regression on the differenced series using 2 lagged differences and the specified deterministic terms in c. The resulting statistic, -1.18653, represents the t-ratio of the coefficient gamma associated with the first lag of the original series. This value tests the null hypothesis that the series contains a unit root against the alternative of stationarity, incorporating the estimated standard error 0.0329132 over 9 observations.

Dickey-Fuller unit-root statistic — `dickey_fuller_test`

The Dickey-Fuller tau statistic: the t-ratio of gamma in the regression Delta x_t = gamma x_{t-1} (+ constant, + trend) + e_t. The statistic only: compare it with Dickey-Fuller critical values.

$$\Delta x_t = \alpha + \gamma x_{t-1} + e_t, \quad \tau = \hat{\gamma}/\mathrm{se}(\hat{\gamma})$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`regression`	in	deterministic terms: c = constant (default), ct = constant and trend, n = none (optional)	—
`tau`	out	the Dickey-Fuller statistic gamma/se	—
`gamma`	out	the coefficient of x_{t-1}	—
`se`	out	its standard error	—
`nobs`	out	observations in the regression (n - 1)	—

Applies when: the series is not constant; default deterministic terms: a constant; no lagged differences (see augmented_dickey_fuller_test).

```
$ time-series solve dickey_fuller_test "x=[10.2, 10.8, 10.5, 11.3, 11, 11.9, 11.6, 12.4, 12, 12.9]" regression=c
tau = -0.9278578506
gamma = -0.2695852535
se = 0.290545856
nobs = 9
  by: Dickey-Fuller unit-root statistic  (least squares of Delta x_t on x_{t-1} (and constant / trend t = 1..n-1))
  regression = c
  assumes: the series is not constant; default deterministic terms: a constant; no lagged differences (see
augmented_dickey_fuller_test)
```

What the worked example means: This calculation determines the Dickey-Fuller unit-root statistic for the input series. It involves regressing the first difference of the series on its lagged value, optionally including a constant or trend term as specified by the regression setting. The resulting statistic, -0.927858, is the ratio of the estimated coefficient -0.269585 to its standard error 0.290546. A more negative value suggests stronger evidence against the presence of a unit root, implying that the series may be stationary.

KPSS stationarity statistic — `kpss_test`

The KPSS statistic for level (c) or trend (ct) stationarity with a Bartlett-kernel long-run variance using a given number of lags. The statistic only: compare it with KPSS critical values.

$$\mathrm{KPSS} = \frac{n^{-2}\sum_{t=1}^{n} S_t^2}{\hat{\sigma}^2_{LR}}, \; \hat{\sigma}^2_{LR} = \frac{1}{n}\Big(\sum e_t^2 + 2\sum_{i=1}^{L}\big(1 - \tfrac{i}{L+1}\big)\sum_t e_t e_{t-i}\Big)$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`nlags`	in	Bartlett lags in the long-run variance	—
`regression`	in	c = level stationarity (default), ct = trend stationarity (optional)	—

symbol		meaning	SI unit
`kpss`	out	the KPSS statistic	—
`long_run_variance`	out	Bartlett long-run variance of the residuals	—

Applies when: 0 <= lags < n; residuals from the mean (c) or a linear trend t = 1..n (ct).

```
$ time-series solve kpss_test "x=[10.2, 10.8, 10.5, 11.3, 11, 11.9, 11.6, 12.4]" nlags=2 regression=c
kpss = 0.4012750353
long_run_variance = 0.8113541667
  by: KPSS stationarity statistic  (eta = sum S_t^2 / n^2 over partial sums S_t of the residuals, divided by the Bartlett long-run
variance)
  regression = c
  assumes: 0 <= lags < n; residuals from the mean (c) or a linear trend t = 1..n (ct)
```

What the worked example means: This calculation determines the KPSS statistic for testing level or trend stationarity. It computes partial sums of residuals from the series [10.2, 10.8, 10.5, 11.3, 11, 11.9, 11.6, 12.4] and divides their squared sum by the Bartlett long-run variance, which depends on the number of lags 2. The resulting statistic 0.401275 should be compared against critical values; a high value suggests non-stationarity, while a low value supports the null hypothesis of stationarity. The long-run variance estimate 0.811354 reflects the autocorrelation structure within the chosen bandwidth.

Transformations and growth (2)

Compound average growth rate of a series — `compound_average_growth_rate`

The compound (geometric) average growth rate per period of a positive series (CAGR when the periods are years), and the total growth from the first to the last value.

$$g = \left(\frac{x_n}{x_1}\right)^{1/(n-1)} - 1$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (positive values)	—
`rate`	out	compound average growth rate per period, as a fraction	—
`rate_percent`	out	the same rate in percent	—
`total_growth`	out	total growth x_n/x_1 - 1, as a fraction	—
`periods`	out	number of periods n - 1	—

Applies when: all values positive; equally spaced periods.

```
$ time-series solve compound_average_growth_rate "x=[100, 110, 125, 133, 150]"
rate = 0.1066819197
rate_percent = 10.66819197
total_growth = 0.5
periods = 4
  by: Compound average growth rate of a series  ((x_n / x_1)^(1/(n-1)) - 1)
  assumes: all values positive; equally spaced periods
```

What the worked example means: This calculation determines the geometric mean growth rate for each time interval in the series. By comparing the final value to the initial value over the total number of periods, it derives a single percentage figure that describes the consistent exponential trend required to produce the observed change. The resulting rate of 10.6682 indicates how much the quantity grew on average per step, while the total accumulated growth over the entire span is 0.5.

Regular and seasonal differencing — `differencing`

Difference a series d times (and seasonally D times at lag s): the series that an ARIMA(p, d, q) or seasonal ARIMA model works on.

$$\nabla^d \nabla_s^D x_t = (1 - L)^d (1 - L^s)^D x_t$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`d`	in	number of regular (lag-1) differences, 0..3 (default 1) (optional)	—
`D`	in	number of seasonal differences, 0..2 (default 0) (optional)	—
`s`	in	seasonal lag (needed when D > 0) (optional)	—
`differenced`	out	the differenced series	—
`last`	out	its latest value	—
`length`	out	its length n - d - D s	—

Applies when: d + D >= 1; the series is longer than d + D s.

```
$ time-series solve differencing "x=[3, 5, 9, 15, 23, 33]" d=2
differenced = [2.0,2.0,2.0,2.0]
last = 2
length = 4
  by: Regular and seasonal differencing  (seasonal differences y_t = x_t - x_{t-s} (D times), then regular differences y_t - y_{t-1} (d times))
  assumes: d + D >= 1; the series is longer than d + D s
```

In the worked example, x = [3, 5, 9, 15, 23, 33], d = 2 go through the equation above and give differenced = [2, 2, 2, 2], last = 2, length = 4. It applies only when d + D >= 1; the series is longer than d + D s: outside that it says what is wrong instead of returning a number.

Trend fitting (7)

Exponential trend fit and forecast — `exponential_trend_forecast`

Fit x_t = a e^(b t) (t = 1..n) by least squares on ln x_t, the growth rate per period e^b - 1, and the forecast of period n + h.

$$\ln x_t = \ln a + b\,t, \quad \hat{x}_{n+h} = a\,e^{b(n+h)}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first (positive)	—
`h`	in	periods ahead (default 1) (optional)	—
`a`	out	the scale a	—
`b`	out	the exponent rate b	—
`growth_rate`	out	e^b - 1 per period	—
`forecast`	out	a e^(b (n + h))	—

Applies when: positive values; least squares on the logarithms (not on the values).

```
$ time-series solve exponential_trend_forecast "x=[100, 112, 124, 140, 155]" h=1
a = 89.64949872
b = 0.1099653413
growth_rate = 0.1162393824
forecast = 173.4170967
  by: Exponential trend fit and forecast  (ordinary least squares of ln x_t on (1, t))
  assumes: positive values; least squares on the logarithms (not on the values)
```

What the worked example means: This calculation fits an exponential trend to the series by applying ordinary least squares to the natural logarithm of each value against its time index. The process transforms the curved relationship into a straight line, making it possible to estimate the base scale 89.6495 and the continuous growth parameter 0.109965. These parameters give the percentage increase per period, which is 0.116239, and project the next value as 173.417. This method assumes that the relative change remains constant over time.

Linear trend fit and forecast — `linear_trend_forecast`

Fit the least-squares trend line x_t = a + b t (t = 1..n) and forecast period n + h.

$$\hat{x}_t = a + b\,t, \quad \hat{x}_{n+h} = a + b(n+h)$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`h`	in	periods ahead to forecast (default 1) (optional)	—
`slope`	out	trend per period b	—
`intercept`	out	a (the trend value at t = 0)	—
`forecast`	out	a + b (n + h)	—
`r_squared`	out	coefficient of determination	—

Applies when: t = 1..n; the series is not constant.

```
$ time-series solve linear_trend_forecast "x=[12, 15, 17, 20, 22, 25]" h=2
slope = 2.542857143
intercept = 9.6
forecast = 29.94285714
r_squared = 0.9969792322
  by: Linear trend fit and forecast  (ordinary least squares of x_t on (1, t))
  assumes: t = 1..n; the series is not constant
```

What the worked example means: This calculation determines the best-fit straight line for the time series data using ordinary least squares. The resulting slope, 2.54286, represents the average change in the series value per period, while the intercept, 9.6, indicates the theoretical starting value at time zero. Based on these parameters, the model forecasts the value for the next periods ahead as 29.9429, with the fit quality measured by the coefficient of determination, 0.996979.

Linear trend with seasonal dummy variables — `trend_seasonal_regression`

Fit x_t = a + b t + (effect of t's season) by least squares with seasonal dummies (the first season is the baseline, effect 0; the series starts in season 1) and forecast period n + h.

$$x_t = a + bt + \sum_{j=2}^{m} \gamma_j D_{j,t} + e_t$$

symbol		meaning	SI unit
`x`	in	the series, oldest first, starting in season 1	—
`m`	in	season length	—
`h`	in	periods ahead (default 1) (optional)	—
`forecast`	out	the fitted value at t = n + h	—
`intercept`	out	a	—
`slope`	out	b per period	—
`season_effects`	out	the m seasonal effects (season 1 = 0)	—

Applies when: n >= m + 3; x_1 is in season 1; season 1 is the baseline.

```
$ time-series solve trend_seasonal_regression "x=[30, 21, 29, 31, 40, 30, 39, 41, 50, 39, 48, 51]" h=1 m=4
forecast = 59.25
intercept = 27.96875
slope = 2.40625
season_effects = [0.0,-12.406249999999996,-6.145833333333334,-6.21875]
  by: Linear trend with seasonal dummy variables  (ordinary least squares on (1, t, D_2, ..., D_m))
  assumes: n >= m + 3; x_1 is in season 1; season 1 is the baseline
```

What the worked example means: This calculation determines the predicted value at time n plus h using ordinary least squares on a model containing a linear time trend and seasonal dummy variables. The intercept 27.9688 represents the expected value in the baseline season when time is zero, while the slope 2.40625 indicates the average change per period. Seasonal effects are estimated relative to the first season, which serves as the reference with an effect of zero. The final forecast combines these components to project future values based on the observed historical pattern.

Mann-Kendall trend test — `mann_kendall_test`

The Mann-Kendall statistic S, its variance (with the tie correction), the continuity-corrected normal score z and the two-sided p-value for a monotonic trend.

$$S = \sum_{i<j} \operatorname{sgn}(x_j - x_i), \ \operatorname{Var} S = \frac{n(n-1)(2n+5) - \sum t(t-1)(2t+5)}{18}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`s`	out	S = sum_{i<j} sign(x_j - x_i)	—
`var_s`	out	variance of S with tie correction	—
`z`	out	(S - sign S) / sqrt(var S)	—
`p_value`	out	two-sided normal p-value	—

Applies when: n >= 4; ties corrected in the variance; continuity correction of 1 in z.

```
$ time-series solve mann_kendall_test "x=[12, 14, 13, 17, 16, 19, 25, 21, 24]"
s = 28
var_s = 92
z = 2.81494459
p_value = 0.004878564475
  by: Mann-Kendall trend test  (count concordant minus discordant pairs; normal approximation)
  assumes: n >= 4; ties corrected in the variance; continuity correction of 1 in z
```

What the worked example means: This calculation determines if there is a statistically significant monotonic trend in the time series data. It computes the Kendall score by counting concordant minus discordant pairs among all observations, resulting in 28. The variance of this score, adjusted for tied values, is calculated as 92. Using these values, a continuity-corrected normal score z is derived as 2.81494, which produces a two-sided p-value of 0.00487856. A small p-value suggests that the observed trend is unlikely to have occurred by random chance alone.

Quadratic trend fit and forecast — `quadratic_trend_forecast`

Fit x_t = b0 + b1 t + b2 t^2 (t = 1..n) by least squares and forecast period n + h.

$$\hat{x}_t = b_0 + b_1 t + b_2 t^2$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`h`	in	periods ahead (default 1) (optional)	—

symbol		meaning	SI unit
`b0`	out	constant	—
`b1`	out	coefficient of t	—
`b2`	out	coefficient of t^2	—
`forecast`	out	the trend value at t = n + h	—

Applies when: t = 1..n; n >= 4.

```
$ time-series solve quadratic_trend_forecast "x=[3, 5, 9, 15, 22, 32]" h=1
b0 = 2.8
b1 = -0.7285714286
b2 = 0.9285714286
forecast = 43.2
  by: Quadratic trend fit and forecast  (least squares (QR) on (1, t, t^2))
  assumes: t = 1..n; n >= 4
```

What the worked example means: This calculation determines the best-fit parabolic curve for the time series data using least squares regression. By fitting the model to the historical values, it derives three key parameters: the constant term 2.8, the linear coefficient -0.728571, and the quadratic coefficient 0.928571. These coefficients define the shape of the trend line, allowing for the prediction of future values based on the observed curvature in the data.

Random-walk-with-drift (drift method) forecast — `drift_forecast`

The drift method: the average change per period (x_n - x_1)/(n - 1) and the forecast x_n + h x drift.

$$\hat{x}_{n+h} = x_n + h\,\frac{x_n - x_1}{n - 1}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`h`	in	periods ahead (default 1) (optional)	—
`drift`	out	average change per period	—
`forecast`	out	x_n + h drift	—

Applies when: n >= 2.

```
$ time-series solve drift_forecast "x=[100, 104, 103, 109, 112]" h=3
drift = 3
forecast = 121
  by: Random-walk-with-drift (drift method) forecast  (drift = (x_n - x_1)/(n - 1))
  assumes: n >= 2
```

What the worked example means: This method calculates the average change per period, known as the drift, using the first and last values of the series. The result shows that the estimated drift is 3. Adding this drift multiplied by the horizon 3 to the final observed value gives a forecast of 121. This assumes the series will continue its historical trend linearly.

Sen's slope (Theil-Sen trend) — `sens_slope`

Sen's robust trend slope (median of all pairwise slopes, t = 1..n) and the intercept median(x) - slope median(t).

$$\hat{\beta} = \operatorname{median}_{i<j}\,\frac{x_j - x_i}{j - i}$$

symbol		meaning	SI unit
`x`	in	the series, oldest first	—
`slope`	out	median of (x_j - x_i)/(j - i), i < j	—
`intercept`	out	median(x) - slope (n+1)/2	—

Applies when: t = 1..n.

```
$ time-series solve sens_slope "x=[12, 14, 13, 17, 16, 19, 25]"
slope = 1.666666667
intercept = 9.333333333
  by: Sen's slope (Theil-Sen trend)  (median of the n(n-1)/2 pairwise slopes)
  assumes: t = 1..n
```

What the worked example means: This calculation determines the Sen's slope, which is the median of all pairwise slopes between points in the series. By using the median instead of the mean, it provides a resistant estimate of the trend direction and magnitude against outliers. The resulting slope of 1.66667 indicates how much the value changes on average for each step forward in time. The intercept of 9.33333 represents the estimated value at the center of the time sequence, calculated by adjusting the median of the data by the trend.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.